

Detection and classification of objects in telescope data using machine learning

Data Science Challenge 2021

Team 3: Alexander Nguyen, Nathan Tran, Sai Poornasree Balamurugan, Severin Field, Rutuja Gurav

Team Mentor: Kerianne Pruett

UC Riverside



Outline



Team Introductions



Background



Results



Summary

Outline



Team Introductions



Background



Results



Summary

Team 3: Rutuja Gurav



Degree: Ph.D. Computer Science

Skills: Machine Learning, Data Mining

Things learned:

- Collaborating with teammates from diverse backgrounds.
- Data wrangling and ML for large astronomy data

With more time:

- More efficient data loading for star-galaxy data
- Better asteroids data preprocessing
- More efficient asteroids dataset generation
- Streaming data loader for asteroids dataset
- Use better, more sophisticated CNNs
- Unsupervised methods for asteroids detection
- NAS for optimal model design

Team 3: Severin Field



Degree: Physics Applied in Computer Science

Applicable Skills: Data structures, python, command line & git basics

Things learned: machine learning concepts and libraries (sklearn, tensorflow, pytorch), model evaluation methods, classical machine learning algorithms, working with large datasets, anaconda

With more time: pursue creating more models for the asteroid dataset, try different subsets of the data, trim & find an automated way to trim out incoherent data

Team 3: Nathan Tran



Degree: Computer Science

Skills: Tensorflow

Things learned: convolutional neural networks, lots of linear algebra

With more time: refine models, test more models

Team 3: Alexander Nguyen



Degree: Chemical Engineering

Skills:

- Intermediate Machine Learning in Python Background from Air Quality Modeling

Things learned:

- What the underlying parts of a neural network are and how to build one
- How to address classification problems and gain experience working with a data science expert
- Working with tensorflow and larger than memory datasets
- Gaining an intuition for how the data on objects in space is collected
- Integrating Dask dataframes for reducing runtime

With more time:

- Learning tensorflow more
- Learning all the different machine learning tools and layers involved in a model
- Spent another week optimizing runtime and learning other techniques for improving accuracy
- Optimize the simple model in our own way

Team 3: Sai Poornasree Balamurugan



Degree: Data Science

Skills: Python, tensorflow

Things learned: Neural networks, image classification

With more time: I would delve deeper into the challenge problems

Outline



Team Introductions



Background

Image Classification
Deep Learning
Convolutional Neural Network

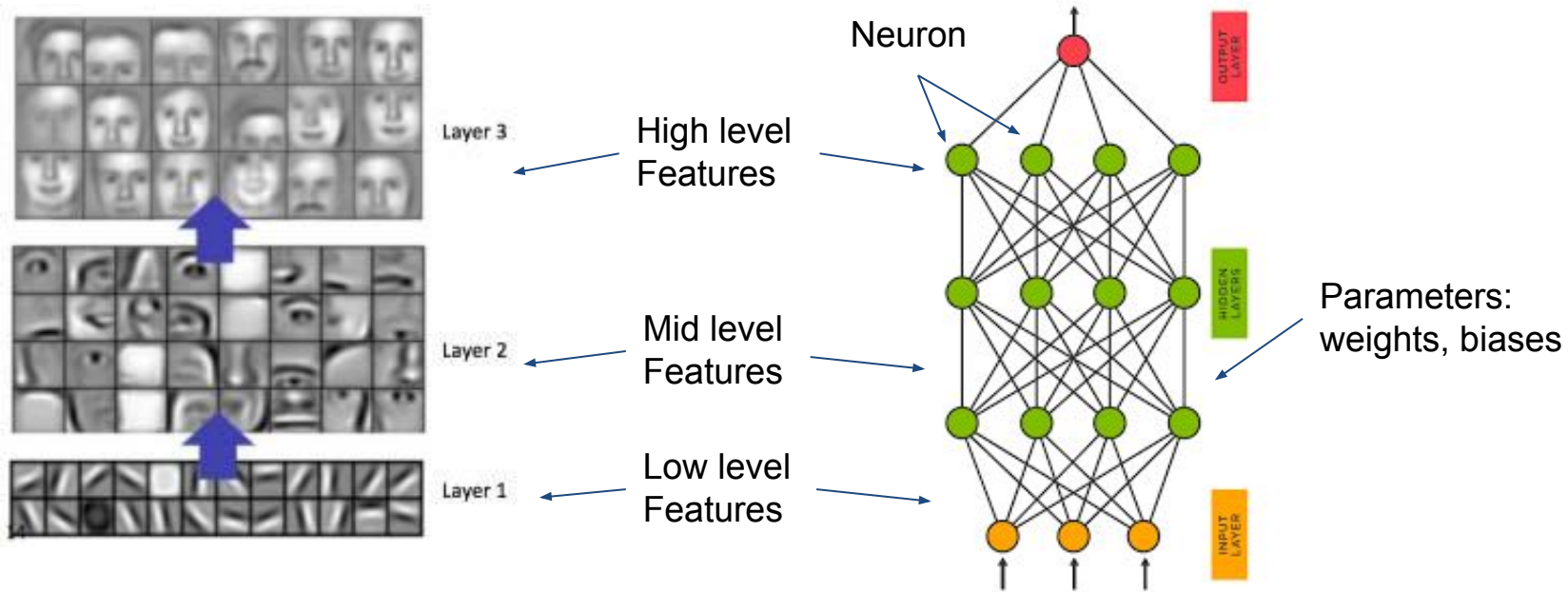


Results



Summary

What is a neural network?



Convolutional Neural Network

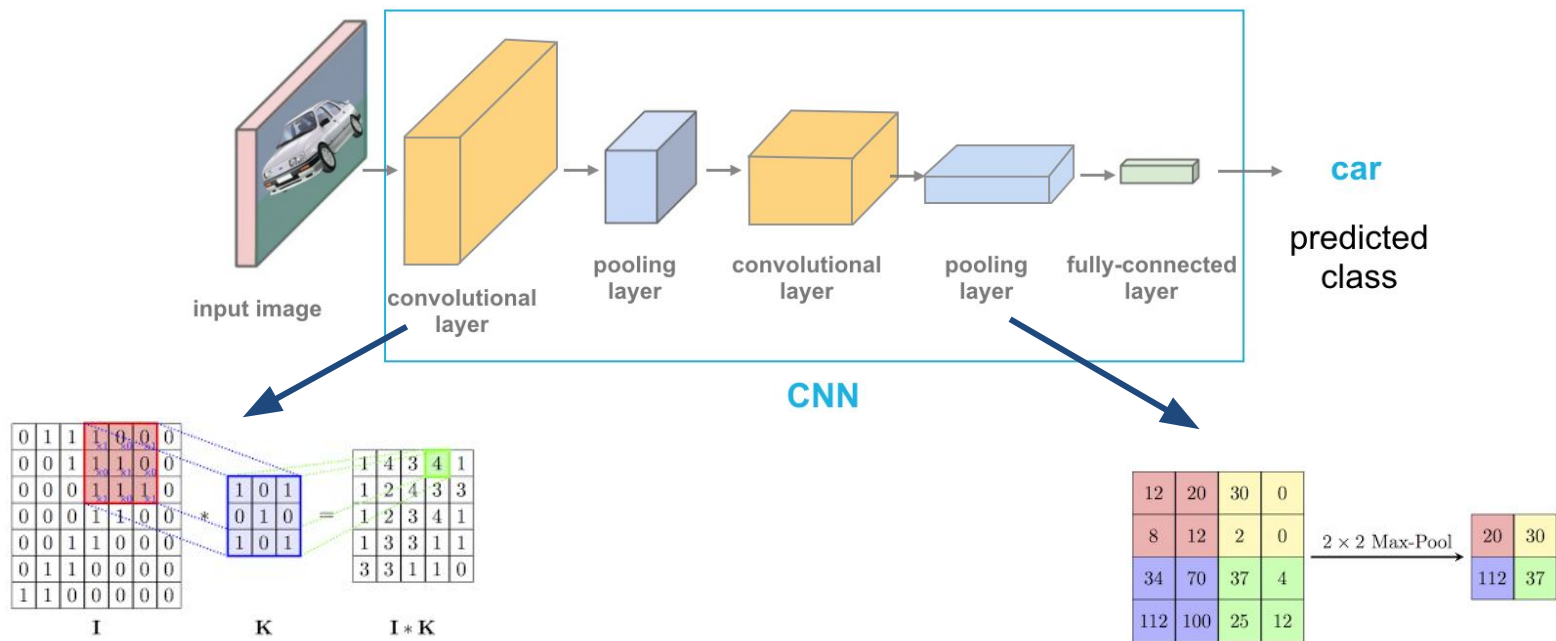
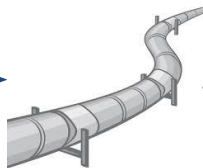


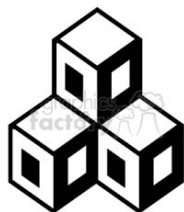
Image Classification Using Tensorflow



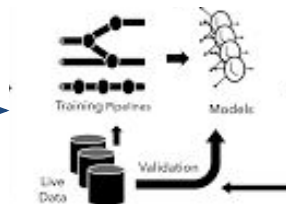
Examine
the data



Build a data
processing
pipeline



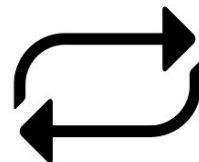
Build the
model



Train the
model



Test the
model



Improve the
model and repeat
process

Outline



Team Introductions



Background

Star-Galaxy Classification:
Exploratory Data Analysis(EDA)
Dataset Preparation

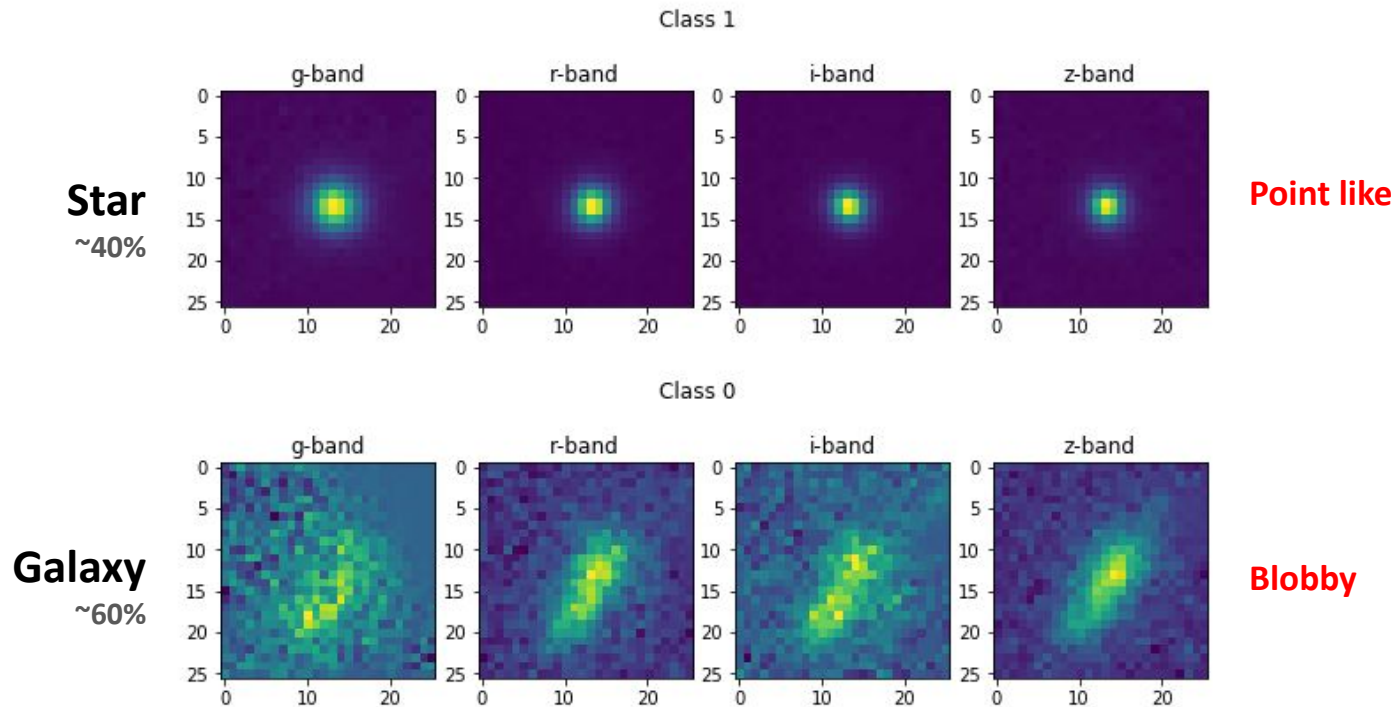


Results

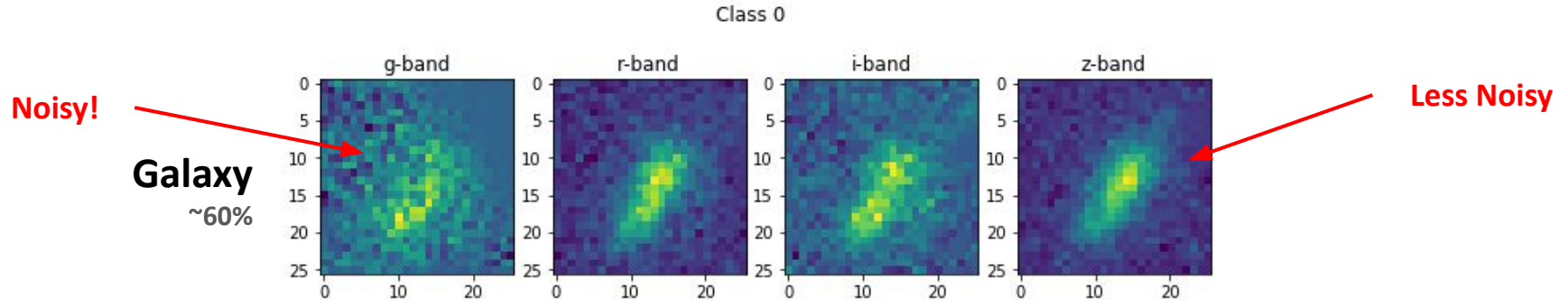
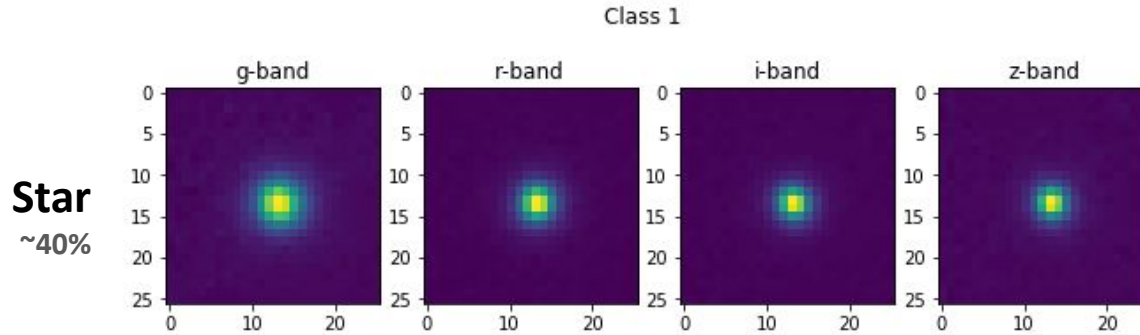
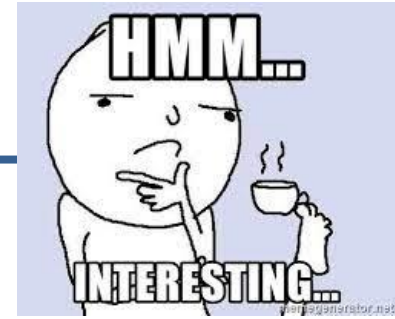


Summary

Star vs Galaxy Classification



Mini-Challenge Problem: Star vs Galaxy Classification



Mini-Challenge Problem: Star vs Galaxy Classification

EDA

Dimensionality Reduction



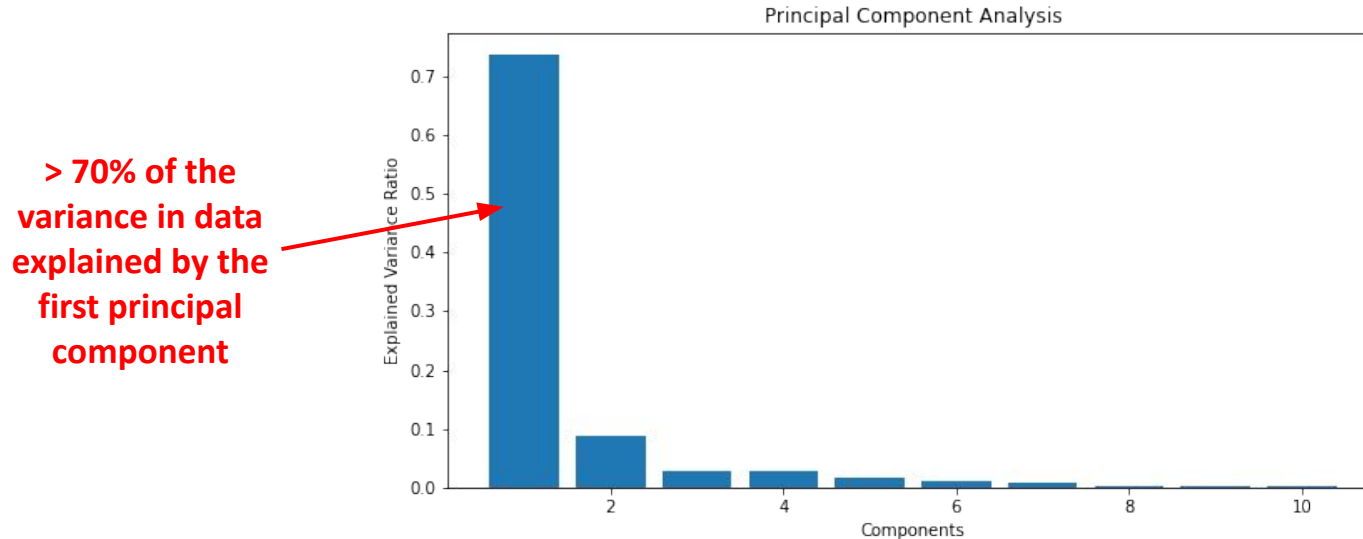
High dimensional data

PCA
→



Low dimensional data

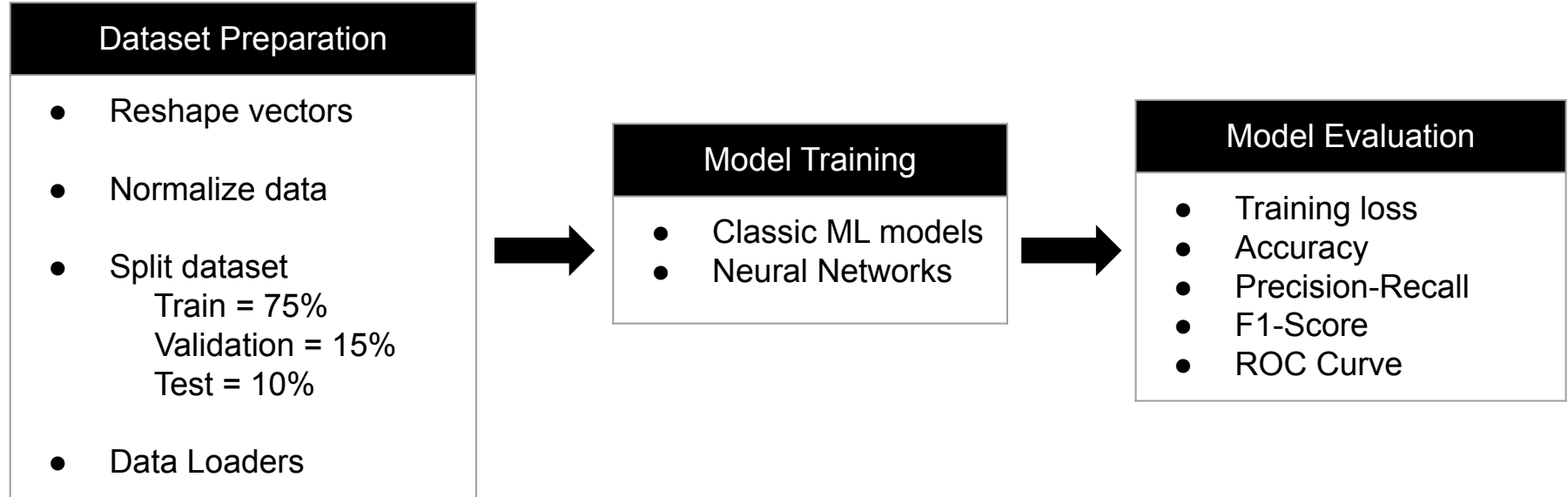
Star vs Galaxy Classification



There is low rank structure in the data.

Star vs Galaxy Classification

Pipeline



Outline



Team Introductions



Background



Results

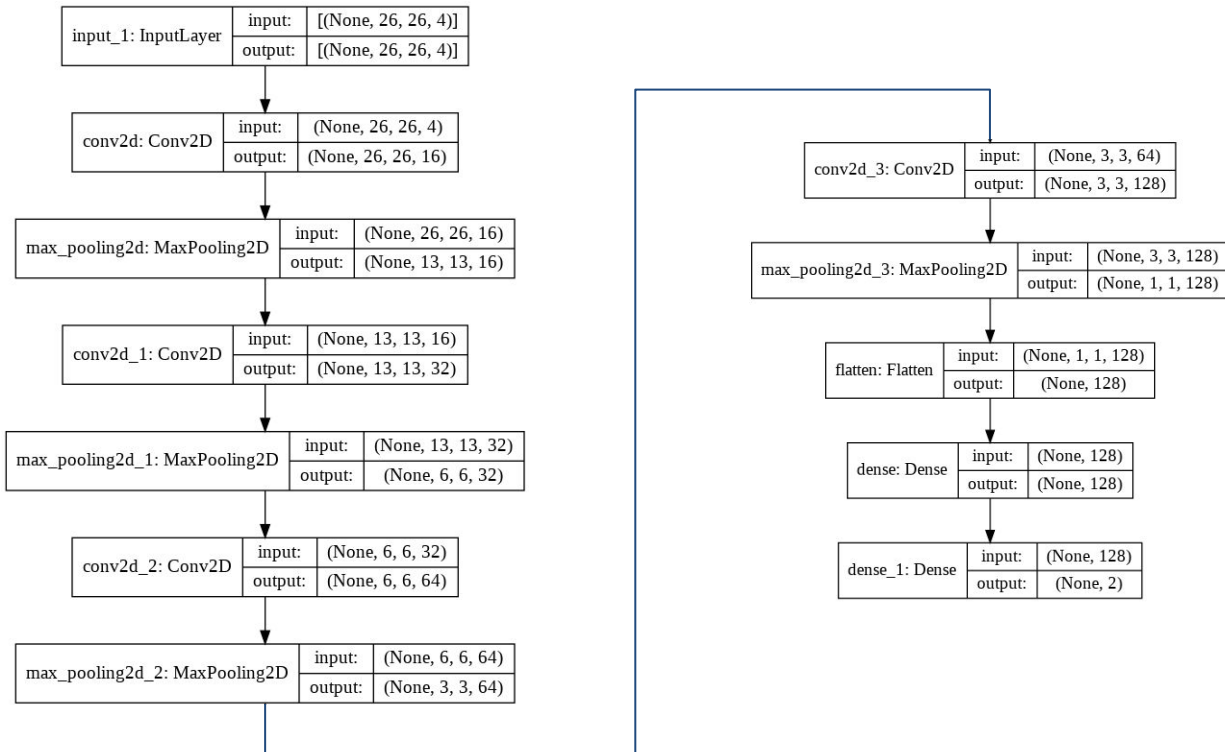
Task	Image Classification
Method	Convolutional Neural Networks
Dataset	Star-Galaxy Classification



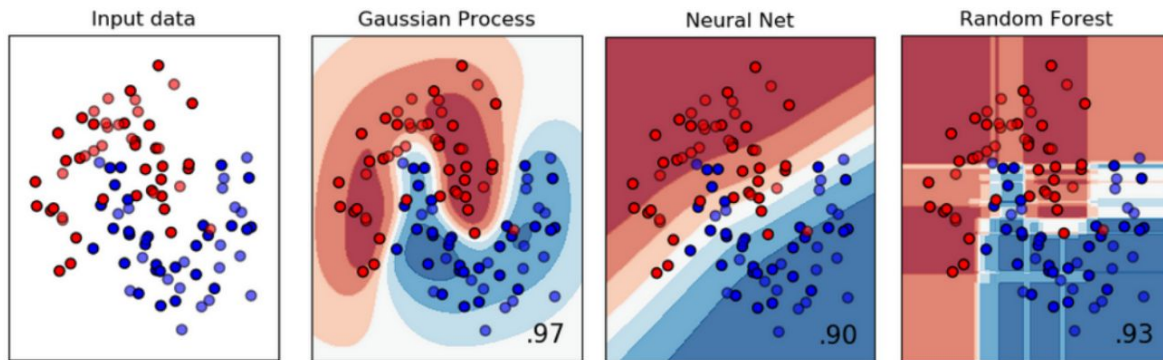
Summary

Mini-Challenge Problem: Star vs Galaxy Classification

Simple Model: a simple CNN

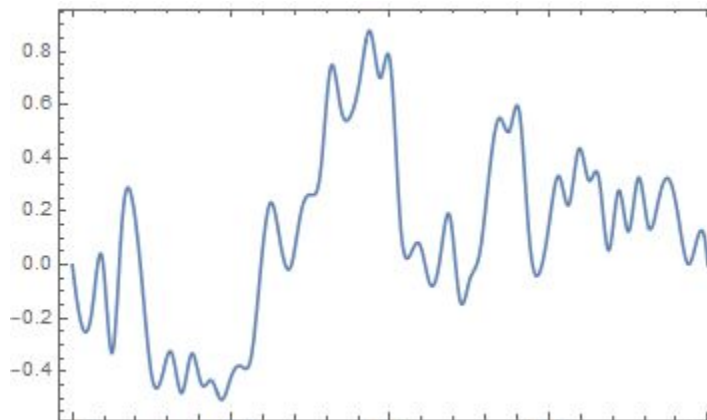
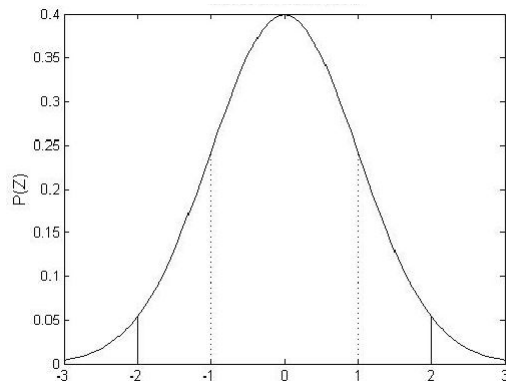


What if we use other classification models instead of convolutional neural networks?



Can we use models which don't use convolutions?

- Different types of probabilistic models tried -
 - Gaussian Process
 - Naive Bayes



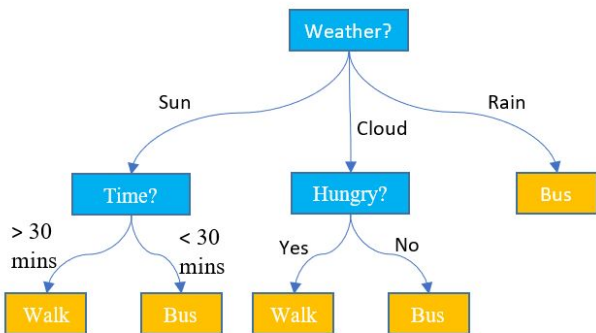
(from sklearn documentation)

- The likelihood of features is assumed to be gaussian:
- Features $X_1, X_2, X_3..$ assumed to be independent
- Label ($y = 0$ or 1)
- Assume $P(Y|X) = P(X|Y) * P(Y) / P(X)$
 - These can all be found from dataset
- Compute $P(\sum x_i | y == 0) = P(x_1 | y==0) * P(x_2 | y==0) * P(x_3 | y==0) \dots$
- Compute $P(\sum x_i | y == 1) = P(x_1 | y==1) * P(x_2 | y==1) * P(x_3 | y==1) \dots$
- Fit a gaussian according to the mean of the data (each feature), then we know its probability function

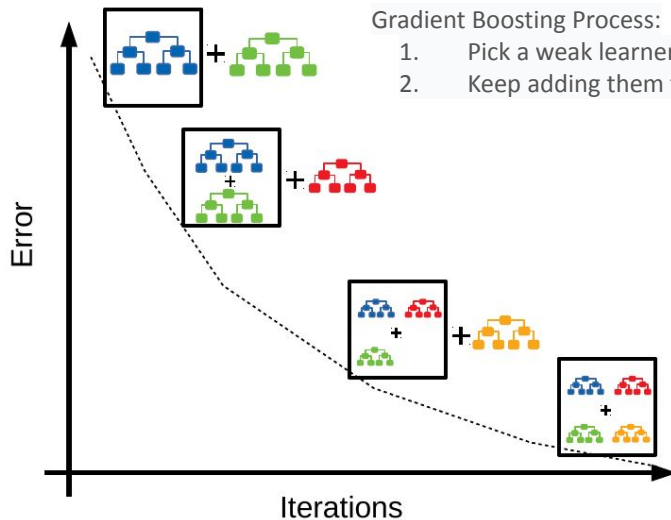
$$P(x_i | y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

Can we use models which don't use convolutions?

Decision Tree



Gradient Boosting Trees



Gradient Boosting Process:

1. Pick a weak learner
2. Keep adding them to the ensemble

Now that we have a trained model, how good is it?

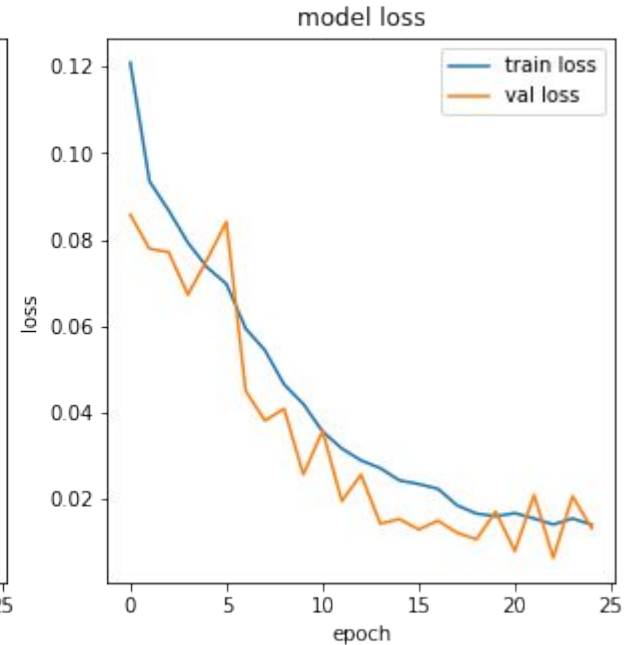
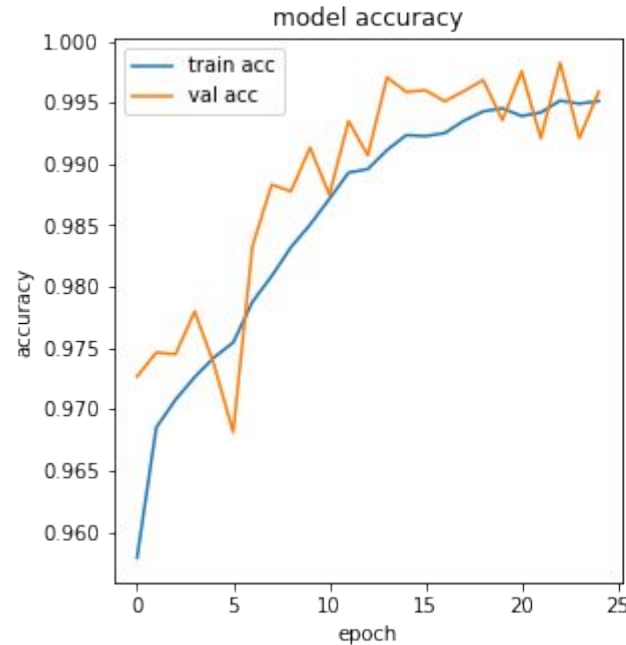
- Training: Epoch-wise loss and accuracy
- Testing: ROC curve, class wise F1 score, precision-recall curve

Simple CNN

Using all bands

Mini-Challenge Problem: Star vs Galaxy Classification

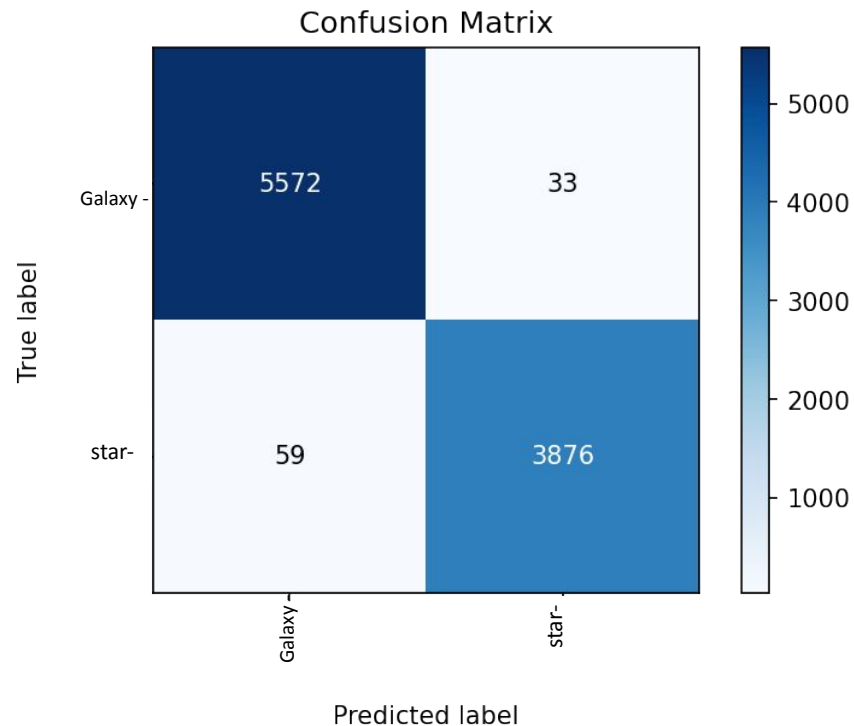
- **Accuracy:** what percentage of predictions made by the model are correct? (higher = better, should increase by epoch)
- **Loss:** how far was the classifier predictions from the ground truth? (lower = better, should decrease by epoch)



Simple CNN

Using all bands

Mini-Challenge Problem: Star vs Galaxy Classification

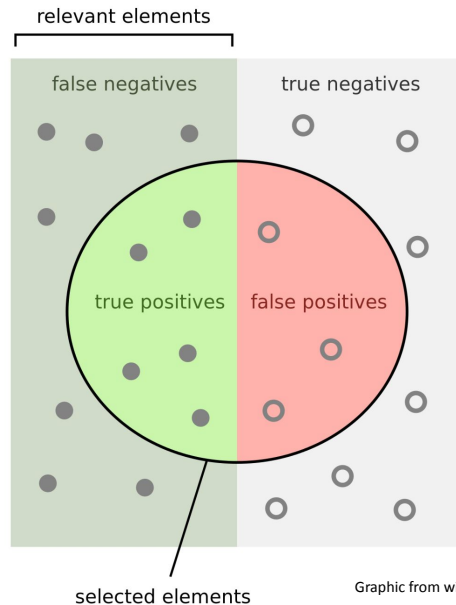
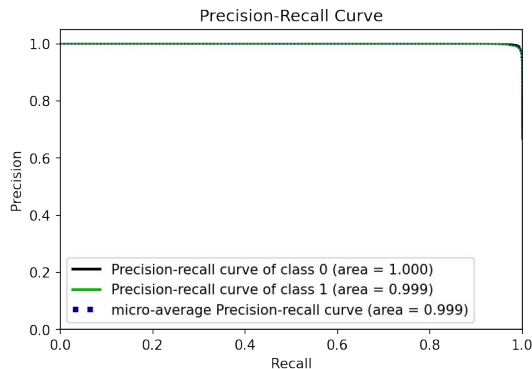


- This is the ideal situation for a confusion matrix
- Very high accuracy
- Low false positives as well as false negatives

Mini-Challenge Problem: Star vs Galaxy Classification

Precision Recall & ROC Curves

- ROC Curve plots the true-positive rate vs an arbitrary false-positive rate as a threshold
- Precision recall plots precision vs. recall



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

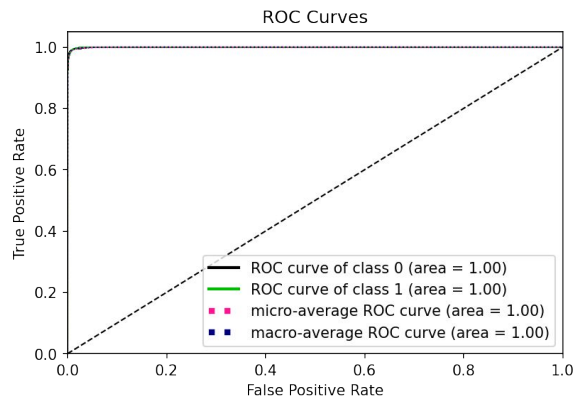
Graphic from wikipedia (https://en.wikipedia.org/wiki/Precision_and_recall)

Simple CNN

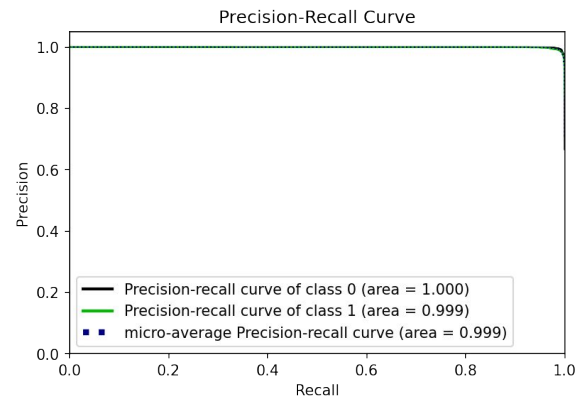
Using all bands

Mini-Challenge Problem: Star vs Galaxy Classification

- FP is a threshold to compare to TP rate, how much better than a random model did we do?

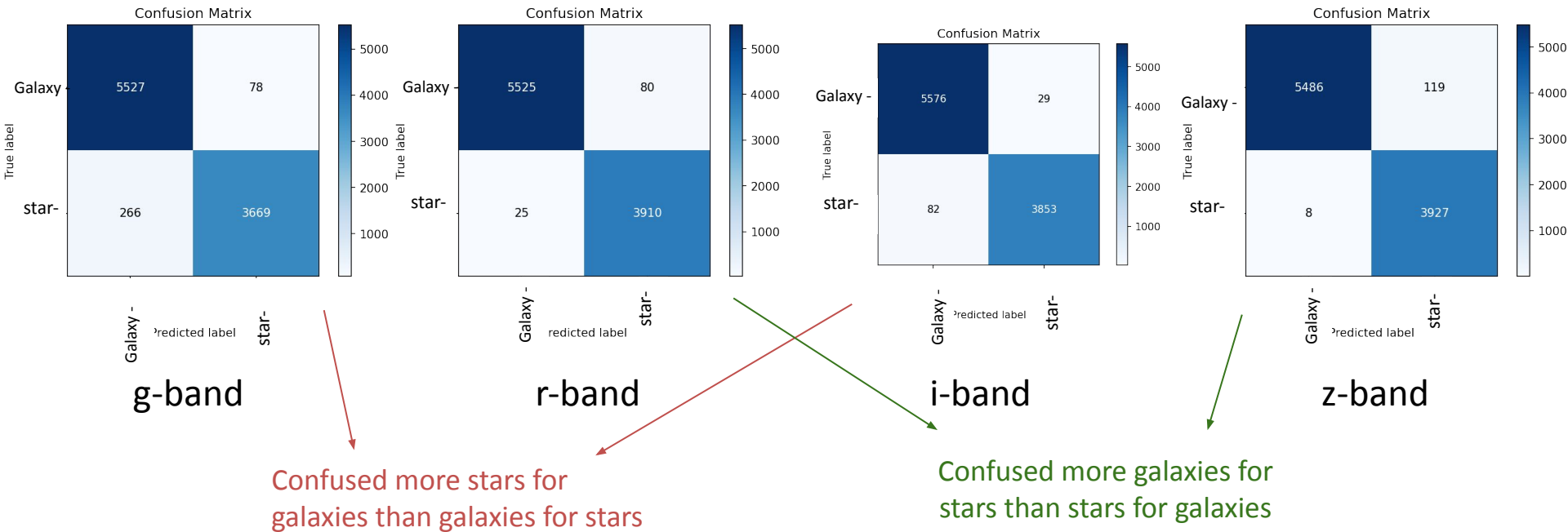


- Our ROC curve for the simple CNN is perfect.



Mini-Challenge Problem: Star vs Galaxy Classification

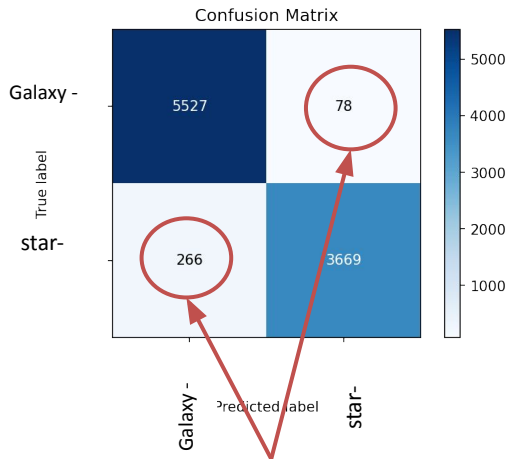
Simple CNN Band-wise Confusion Matrix



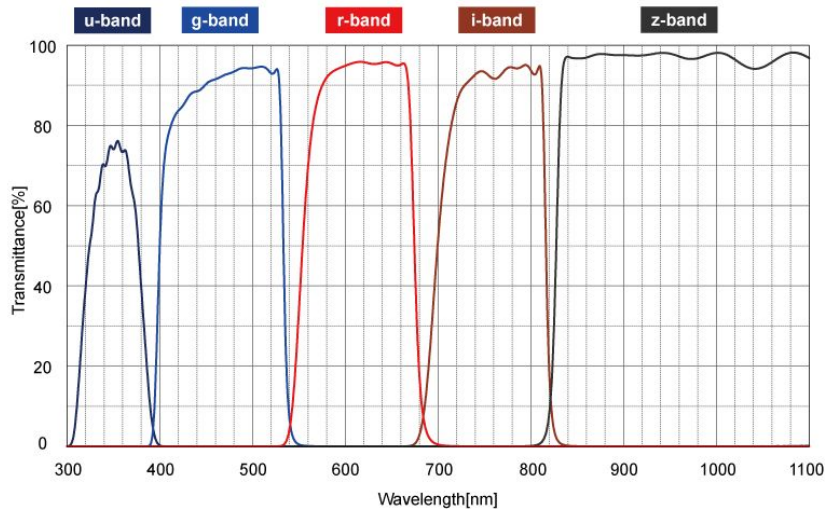
Mini-Challenge Problem: Star vs Galaxy Classification

Simple CNN findings

g-band

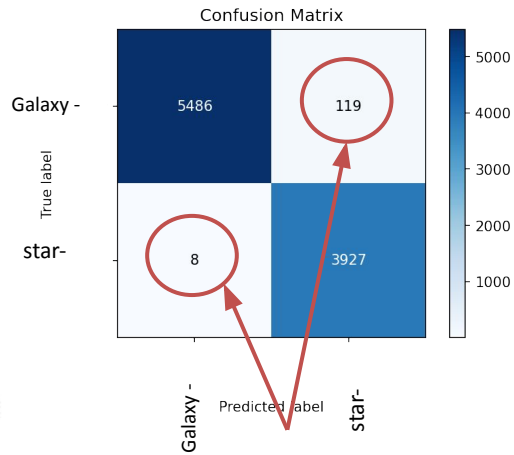


False Negatives
>
False Positives



Source: <https://www.asahi-spectra.com/opticalfilters/sdss.html>

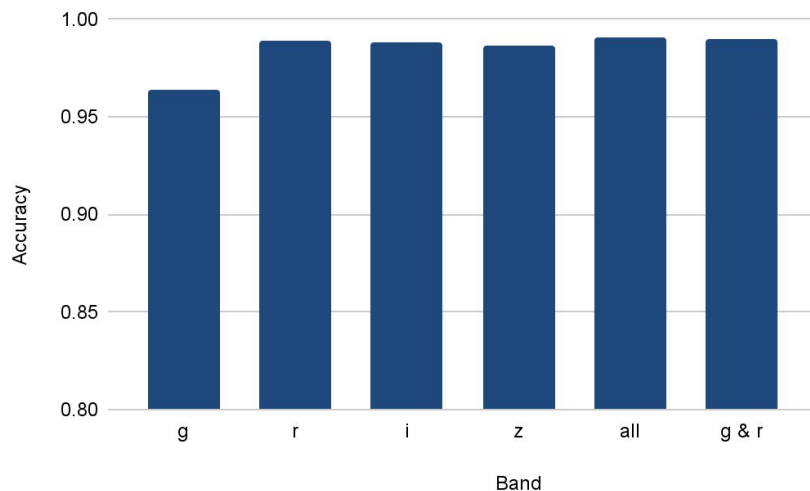
z-band



False Negatives
<
False Positives

Simple CNN: Accuracy by band

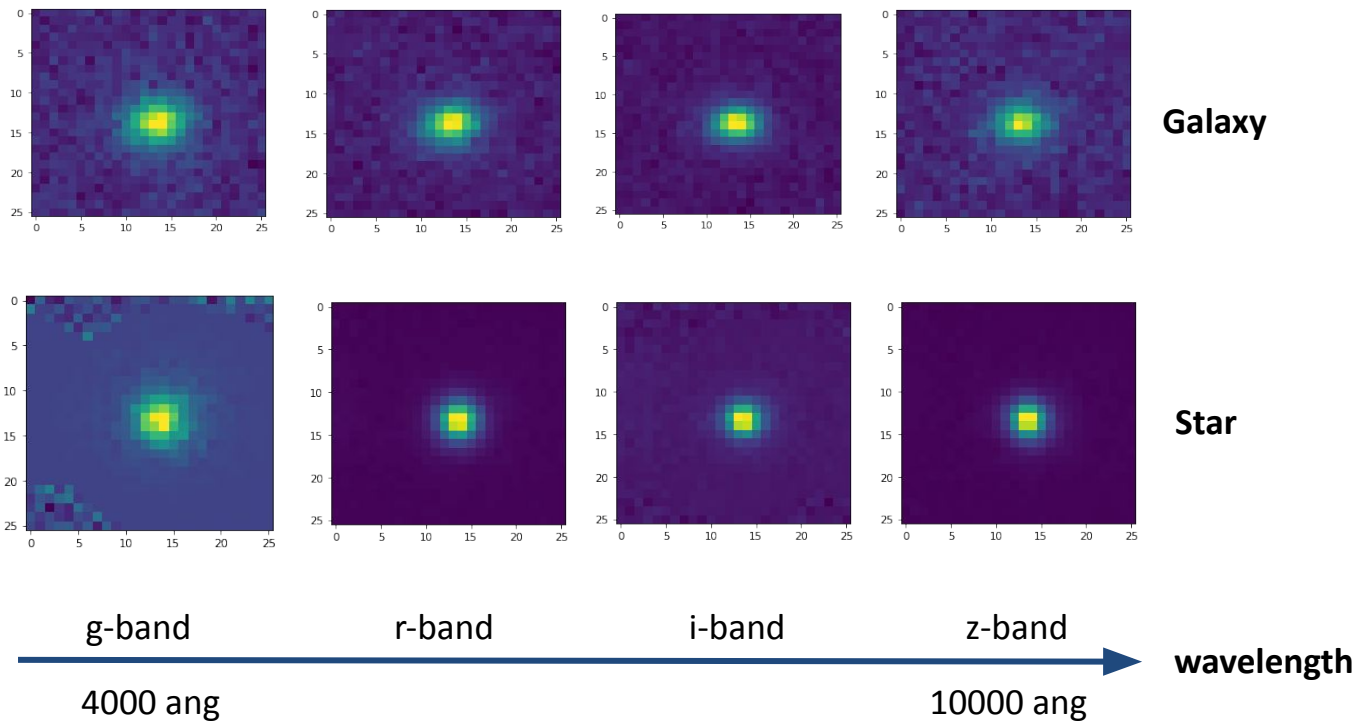
Simple CNN



- G band performs the worst
- All bands combined performs best

Mini-Challenge Problem:
Star vs Galaxy Classification

Hypothesis from performance



Outline



Team Introductions



Background



Results

Task
Method

Image Classification

Alternate models:

→ Gaussian Process

→ Naive Bayes

Dataset

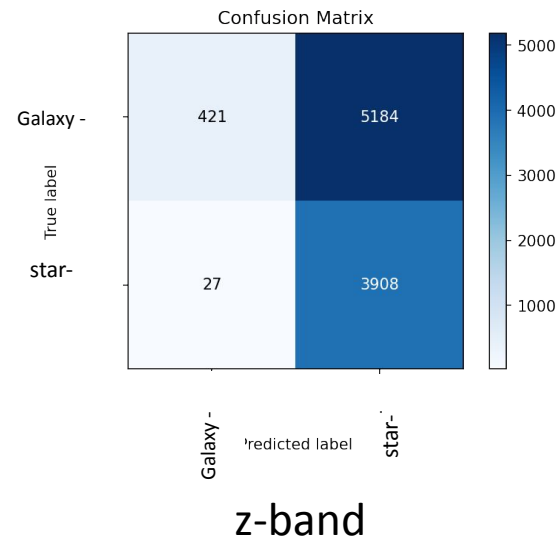
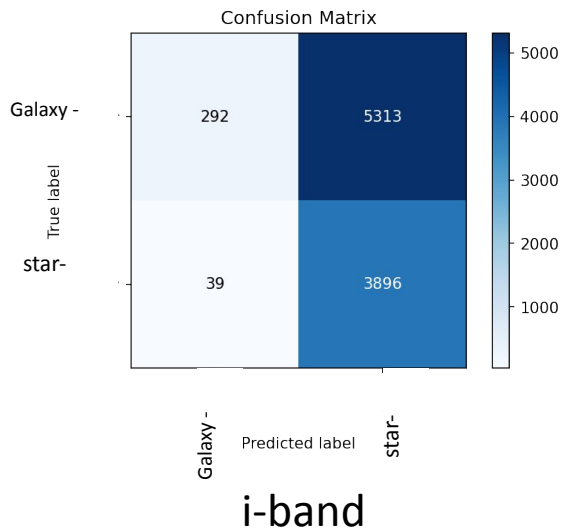
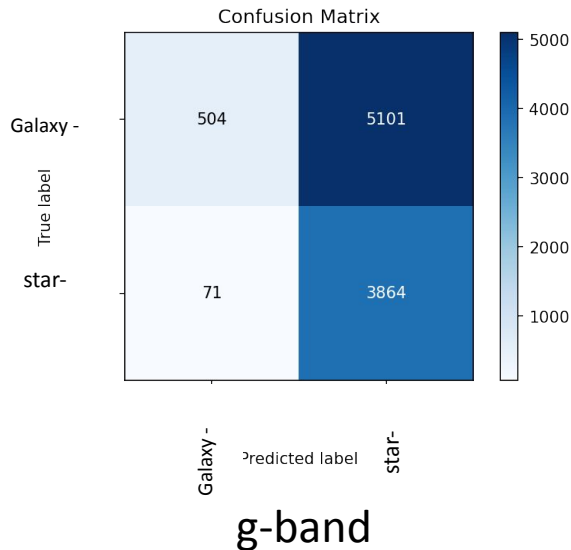
Star-Galaxy Classification



Summary

Mini-Challenge Problem: Star vs Galaxy Classification

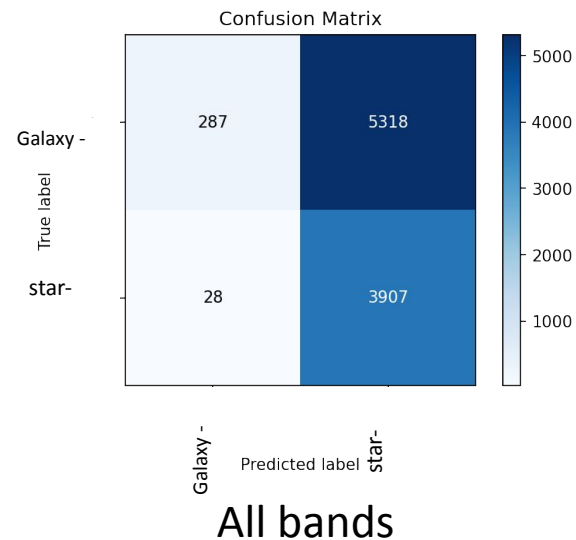
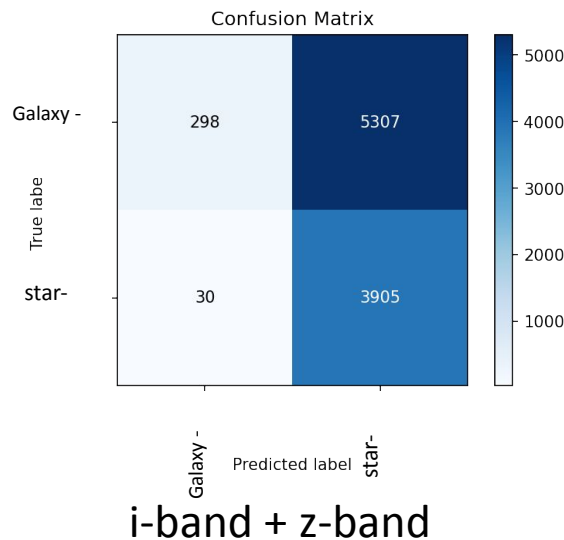
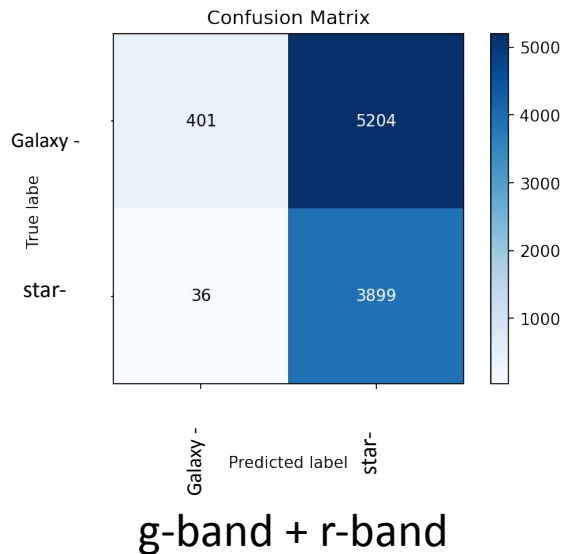
Naive Bayes Model Confusion Matrix



Question: Naive Bayes does not perform well overall! Why?

Mini-Challenge Problem: Star vs Galaxy Classification

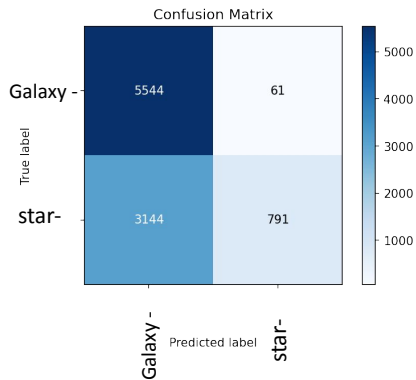
Naive Bayes Model Confusion Matrix



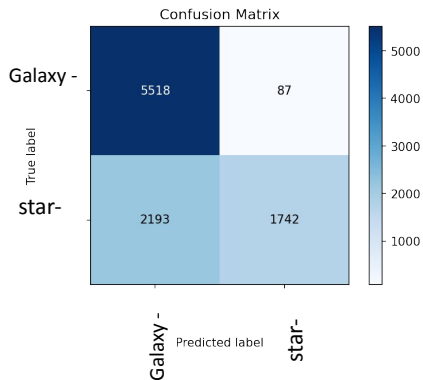
Naive Bayes assumption of feature independence does not hold for this data, it is not a good model for image classification

Mini-Challenge Problem: Star vs Galaxy Classification

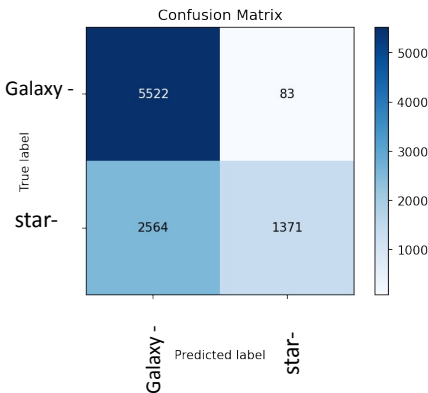
Gaussian Process Confusion Matrix



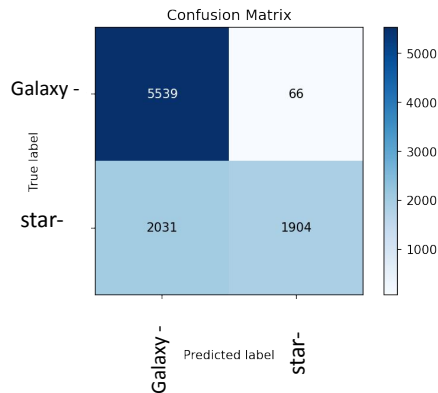
g-band



r-band



i-band



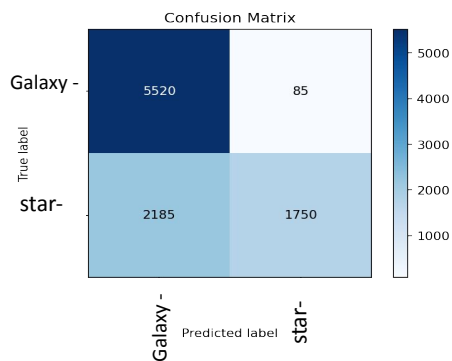
z-band

High number of false negatives!

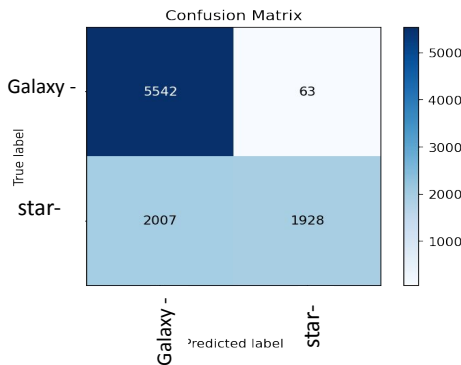
Mini-Challenge Problem: Star vs Galaxy Classification

Gaussian Process Confusion Matrix

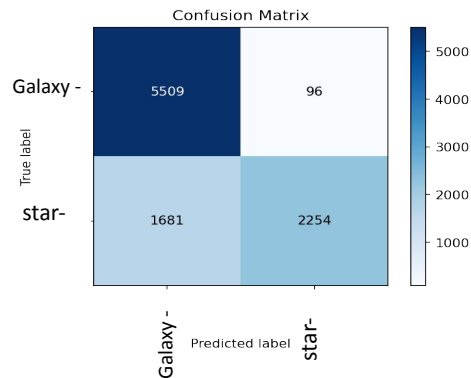
g-band + r-band



i-band + z-band



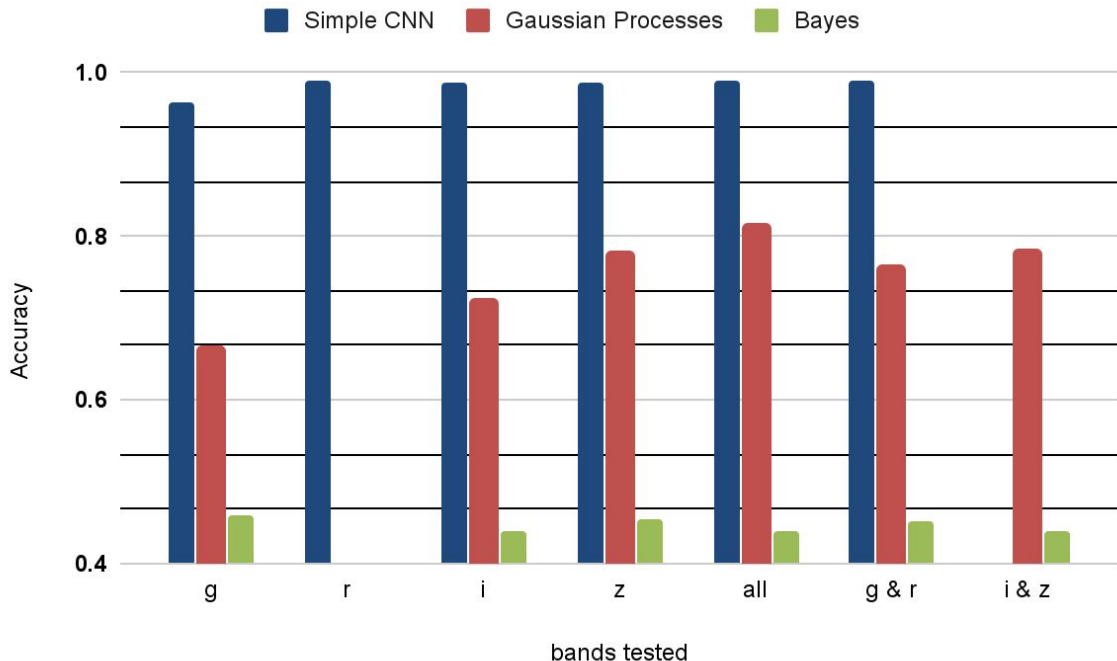
All bands



- Bands 0 and 2 (g & i) were noticeably worse at classifying stars than r and z bands.
- This agrees with the previous findings and the hypothesis that stars are brighter in the r and z bands relative to galaxies which are brighter in g and i.

Mini-Challenge Problem:
Star vs Galaxy Classification

Comparing Simple CNN, GP and Naive Bayes

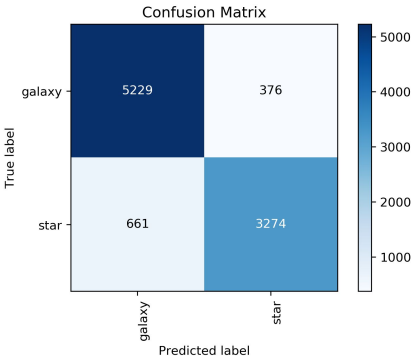


Mini-Challenge Problem:
Star vs Galaxy Classification

Classical ML

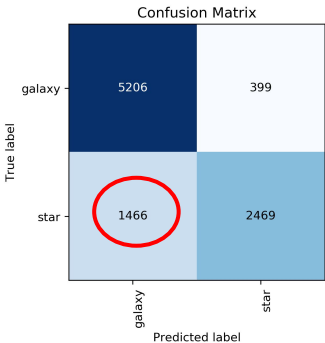
Tree-based Models

g-band performs worst!

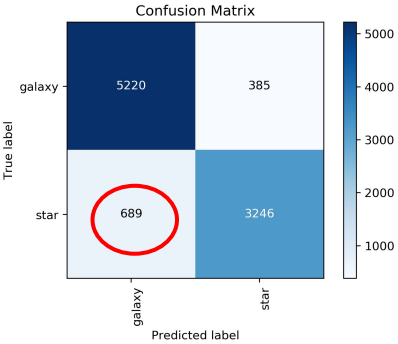
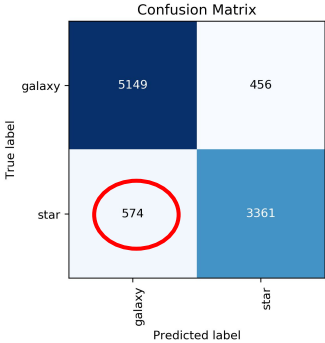


All bands

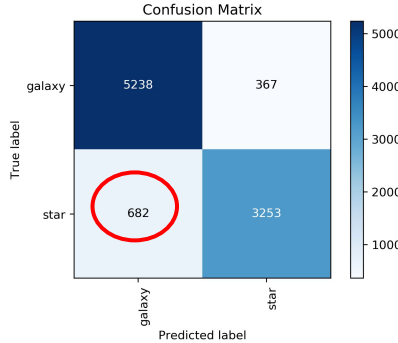
g-band



i-band



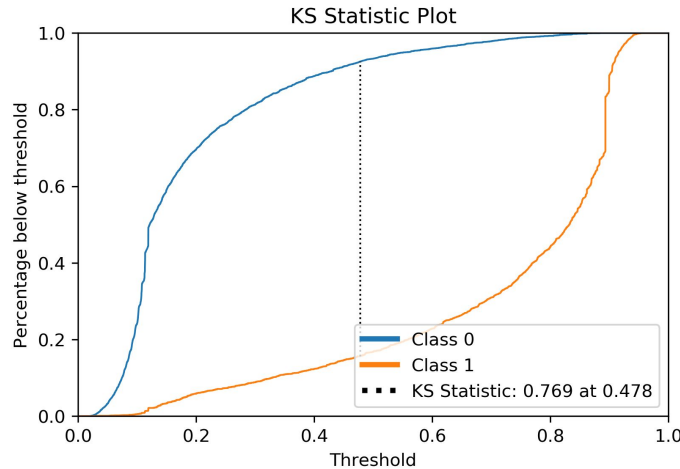
r-band



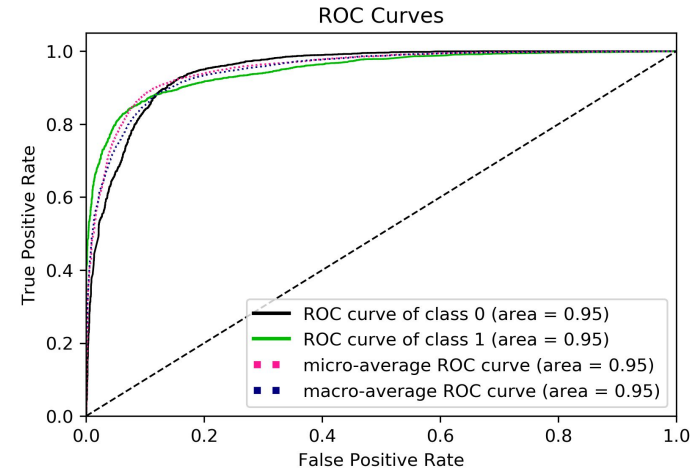
z-band

Mini-Challenge Problem: Star vs Galaxy Classification

Using all bands



**Maximum difference between
TPR and FPR is 0.769 at a
cutoff of 0.47**



**Gradient Boosting Trees
aren't too bad!**

Outline



Team Introductions



Background



Results

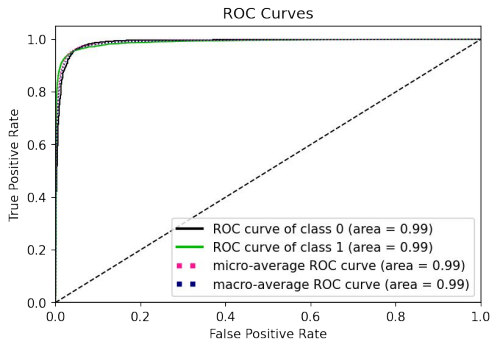
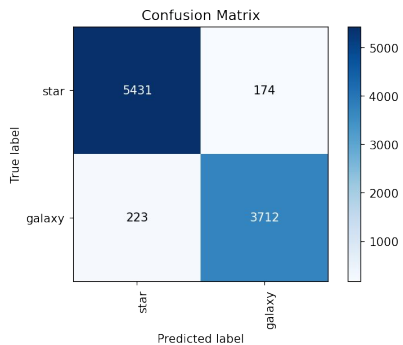
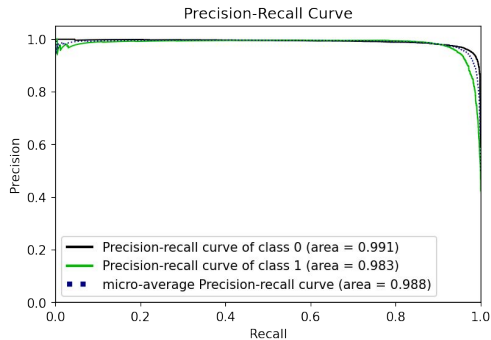
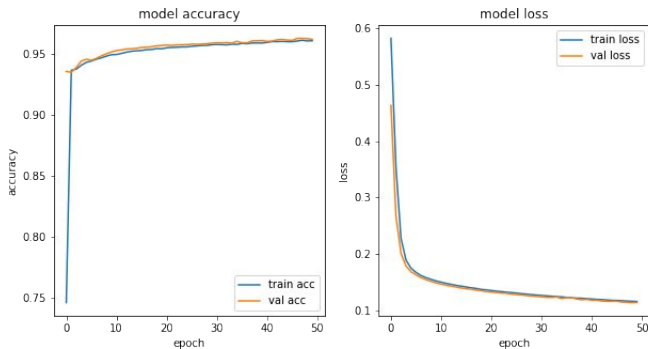
Task	Image Classification
Method	LeNet5
Dataset	Star-Galaxy Classification



Summary

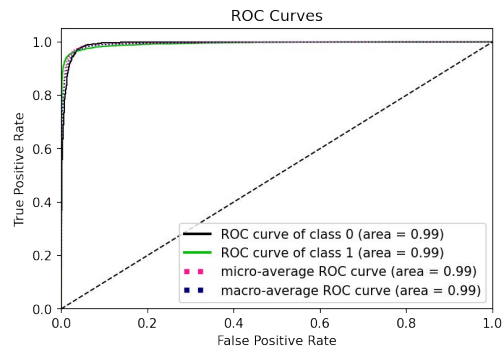
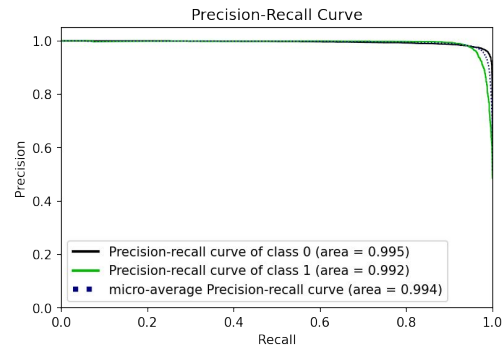
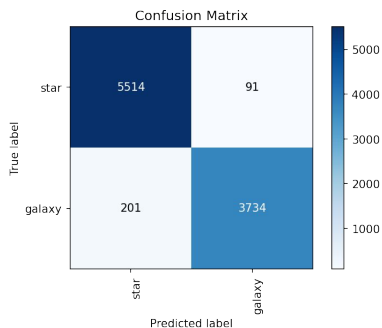
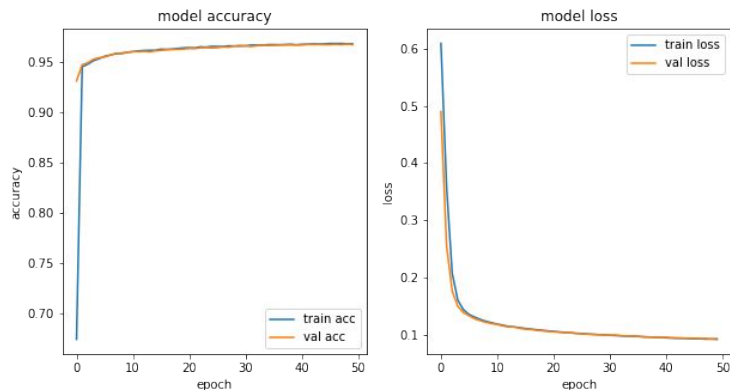
Mini-Challenge Problem: Star vs Galaxy Classification

LeNet5: Band i

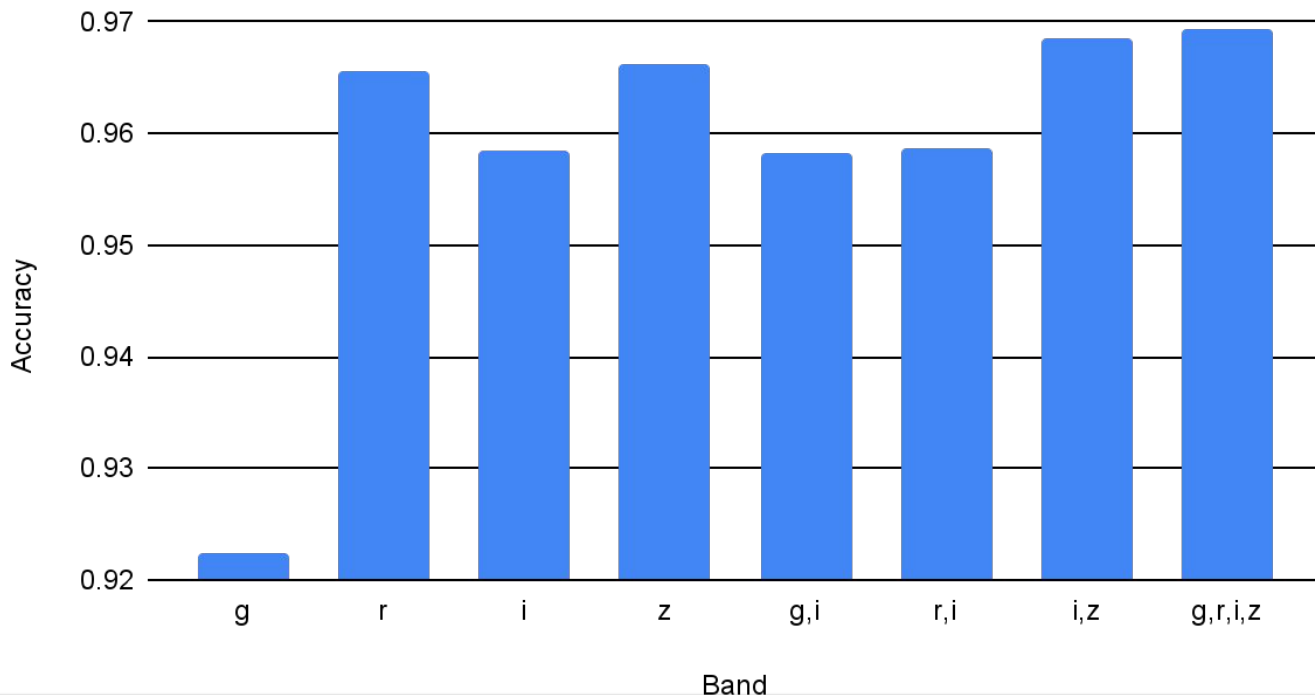


Mini-Challenge Problem: Star vs Galaxy Classification

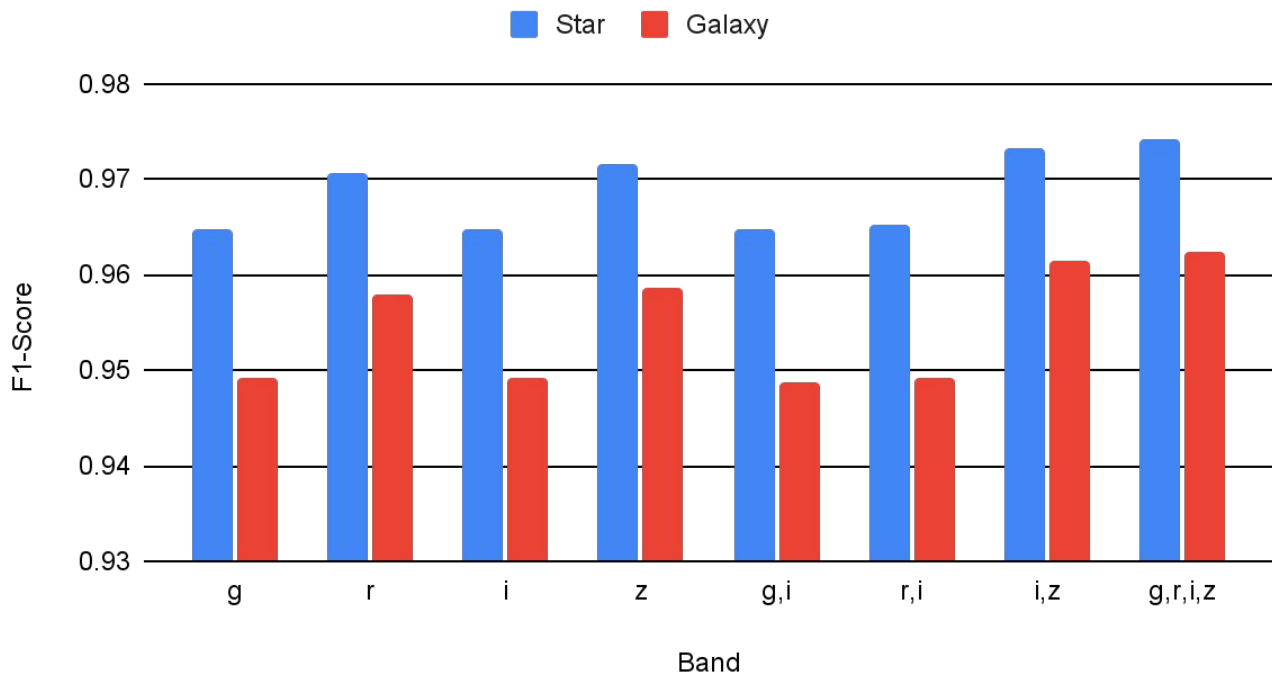
LeNet5: Band g,r,i,z



Accuracy vs. Band



F1-Score vs. Band



Outline



Team Introductions



Background



Results

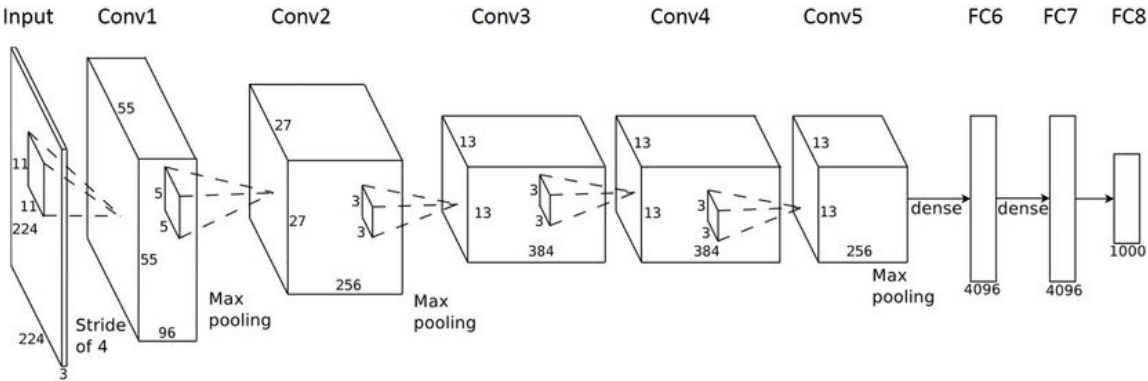
Task	Image Classification
Method	AlexNet
Dataset	Star-Galaxy Classification



Summary

Mini-Challenge Problem:
Star vs Galaxy Classification

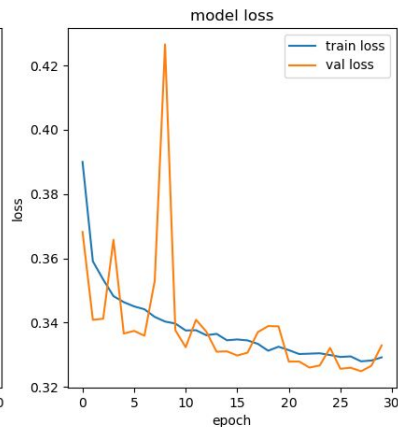
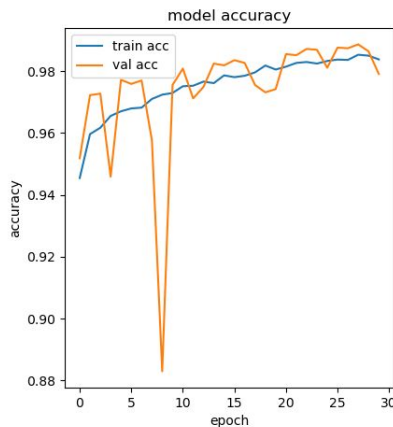
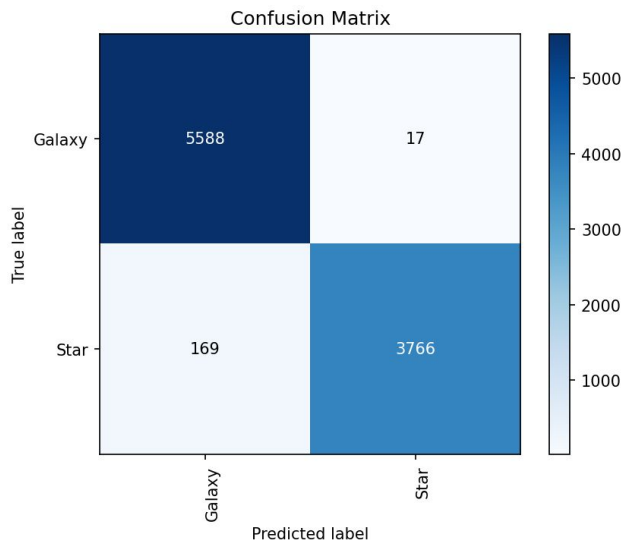
ALEXNET: Intro



Model	Fitting Time (30 Epochs)
ALEXNET	2:38:46
Naive 4 Layer CNN Classifier	0:05:50
Naive 5 Layer CNN Classifier	0:06:56
Naive 6 Layer CNN Classifier	0:11:38
Naive 7 Layer CNN Classifier	0:28:59
Naive 8 Layer CNN Classifier	0:48:49
Gradient Boosting Classifier	0:58:00

Mini-Challenge Problem: Star vs Galaxy Classification

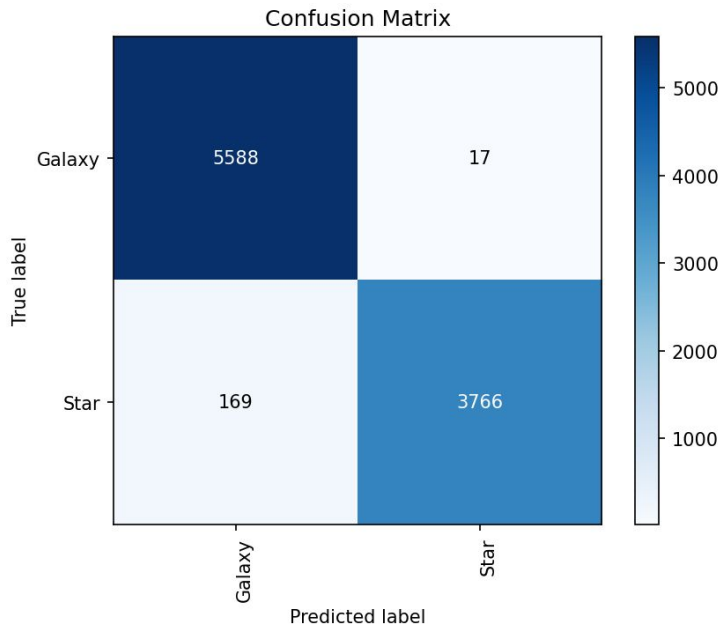
ALEXNET: Evaluation on All Bands



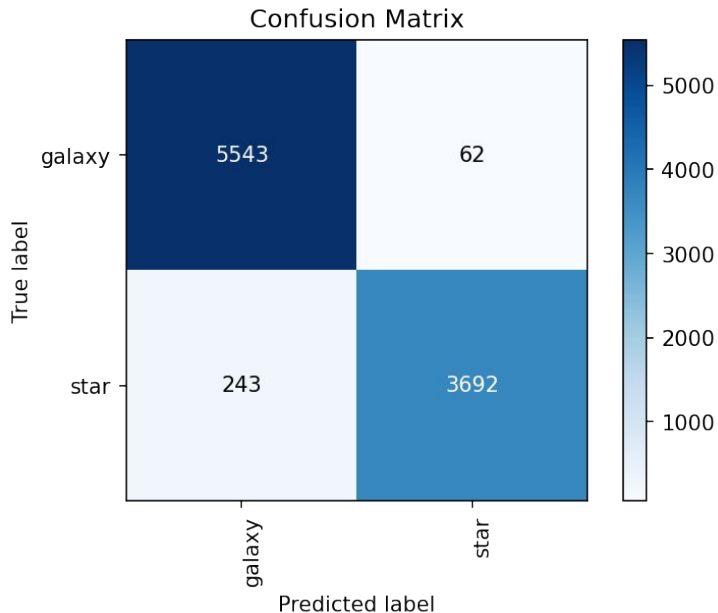
	precision	recall	f1-score	support
Galaxy(class 0)	0.970644433	0.996966994	0.983629643	5605
Star(class 1)	0.995506212	0.957052097	0.975900492	3935
accuracy	0.980503145	0.980503145	0.980503145	0.980503145
macro avg	0.983075322	0.977009545	0.979765068	9540
weighted avg	0.980899265	0.980503145	0.980441571	9540

Mini-Challenge Problem: Star vs Galaxy Classification

ALEXNET: Is it worth the wait?



ALEXNET Confusion Matrix



Simple CNN Confusion Matrix

Outline



Team Introductions



Background

Asteroids Detection:
Exploratory Data Analysis (EDA)
Dataset Preparation



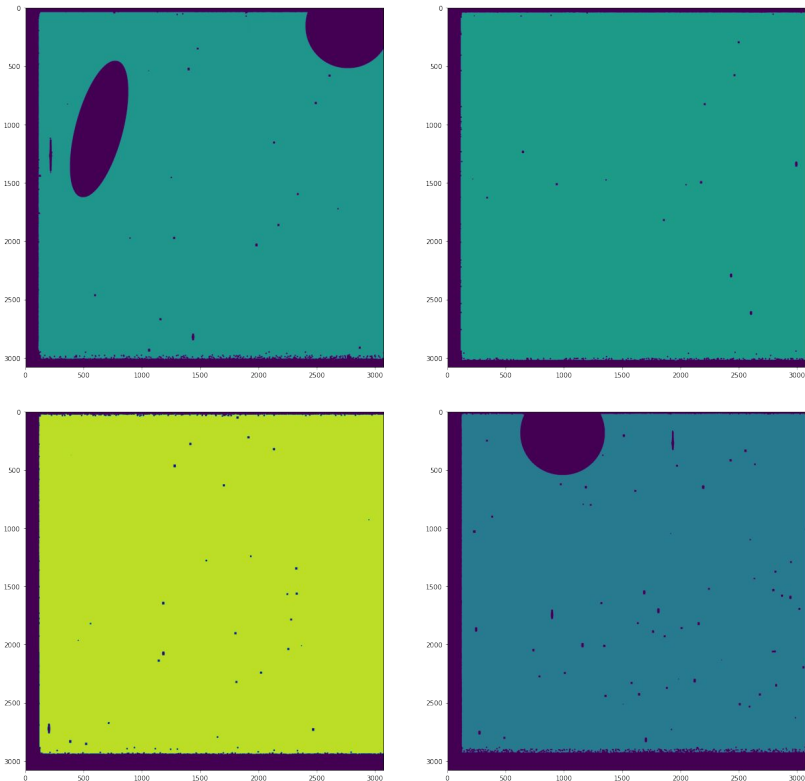
Results



Summary

Challenge Problem:
Asteroids Detection

Understanding Asteroid Data



	0	1	2	3	4	5	6	7	8	9	...	30
0	-4578.137695	-4578.154785	-4578.025879	-4578.212402	-4577.932129	-4577.941895	-4578.103516	-4578.291016	-4578.267578	-4578.139160	...	-4577.9360
1	-4578.185059	-4578.256836	-4578.193359	-4578.171387	-4578.008789	-4578.135742	-4578.353516	-4578.203125	-4577.979482	-4578.235352	...	-4577.9682
2	-4578.129395	-4577.969238	-4577.944824	-4578.210938	-4578.034668	-4578.257324	-4578.070312	-4578.136719	-4578.151367	-4578.334473	...	-4578.2661
3	-4577.977539	-4577.941406	-4577.988281	-4578.147461	-4578.005371	-4578.009277	-4578.274414	-4577.933594	-4578.299805	-4578.021973	...	-4578.0161
4	-4578.093750	-4578.162598	-4578.168945	-4578.098633	-4578.347168	-4577.976562	-4577.936523	-4578.118164	-4578.076172	-4578.350098	...	-4578.0849
...	...	...	...	...	...	...	...	...	...	...	...	...
3075	-4577.989258	-4577.979980	-4578.266113	-4578.352051	-4577.998047	-4577.927734	-4577.883301	-4578.181641	-4578.150391	-4578.188477	...	-4578.3320
3076	-4577.992676	-4578.110352	-4578.314941	-4578.157227	-4577.873535	-4578.217285	-4578.010742	-4577.895508	-4578.216797	-4577.965332	...	-4578.3051
3077	-4578.179688	-4577.875977	-4577.875000	-4578.042480	-4578.174805	-4578.134277	-4578.310547	-4578.256348	-4577.944824	-4577.919434	...	-4578.1997
3078	-4577.875977	-4578.151855	-4578.181641	-4577.938477	-4577.895020	-4578.014648	-4578.147949	-4578.044922	-4577.965820	-4577.878418	...	-4577.8696
3079	-4577.930176	-4578.272461	-4578.096191	-4577.864258	-4578.292480	-4578.022949	-4578.350098	-4578.282227	-4578.246094	-4577.971680	...	-4578.1494

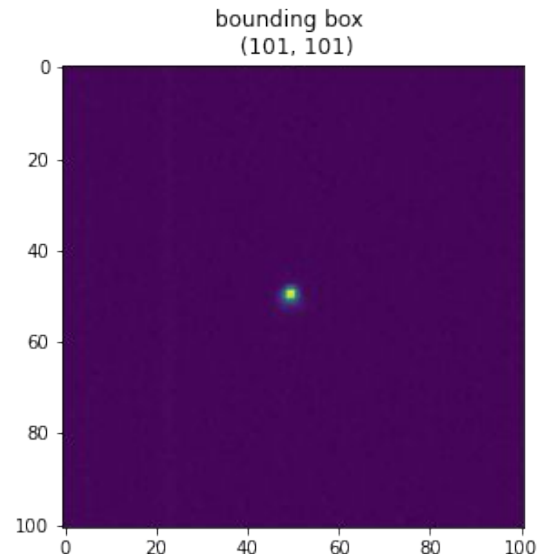
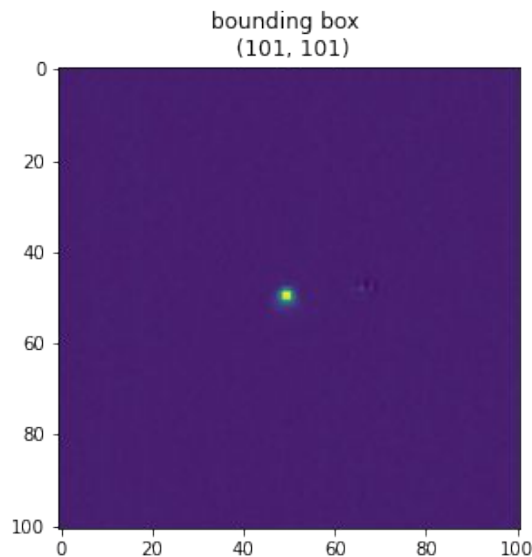
3080 rows x 3072 columns



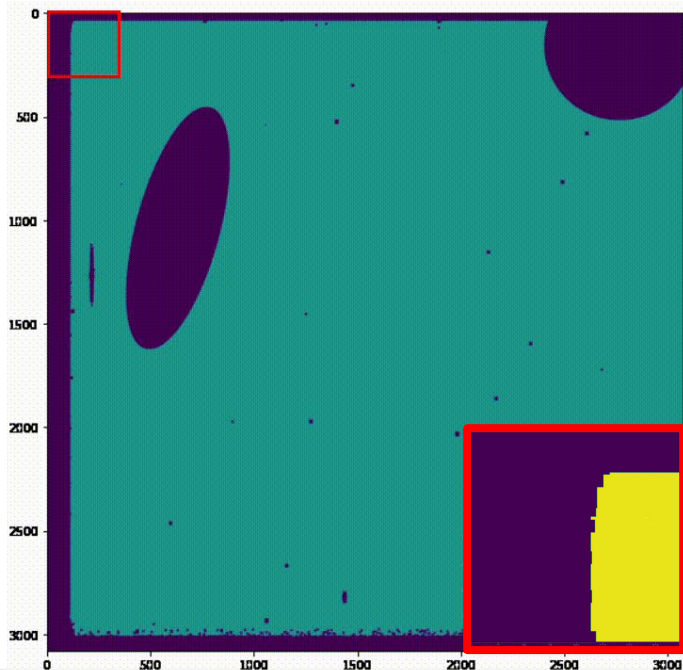
Challenge Problem: Asteroids Detection

Understanding Asteroid Data

- Data contains bounding box locations for 20 asteroids in each difference image
- Each one is a box of around 100x100px
- From the pictures we looked at, asteroid is in the center
- `get_coords()` method returns a tuple of the center of a given bounding box.
- $[(x_{\min} + x_{\max})/2, (y_{\min} + y_{\max})/2]$



- Create dataset in the same format as galstar: dataframe where first column is label followed by vectorized image



*example of a 100x100 image from the larger difference image

- Alex wrote the original data loader that created a list of vectorized images (only negatives/no asteroid)
- Mine instead creates a flattened nparray within current red box → converts to list → adds 1 to beginning of list for asteroid, 0 for none → converts to series → appends to dataframe
- Horizontal rate: 10px, vertical rate: 100px

Challenge Problem: Asteroids Detection

Creating Asteroid Dataset

```
while y_j < amt_rows:
    while x_j < amt_cols:
        output_img = image_array[y_i:y_j, x_i:x_j].flatten()
        is_asteroid = 0 ##Not an asteroid
        print(asteroids)

        for i in asteroids:
            xcoord = get_coords(i[0])[0]
            ycoord = get_coords(i[0])[1]
            if (get_coords(i[0])[0] > x_j) and (get_coords(i[0])[0] < x_j) and (get_coords(i[0])[1] > y_j) and (get_coords(i[0])[1] < y_j):
                is_asteroid = 1
                print(get_coords(i[0])[0])
            xcoord = ( x_j + x_j ) / 2
            ycoord = ( y_j + y_j ) / 2
            if is_asteroid == 0:
                rand = random()
                if rand < .03:
                    output_img = np.insert(output_img, 0, ycoord, axis=0)
                    output_img = np.insert(output_img, 0, xcoord, axis=0)
                    output_img = np.insert(output_img, 0, is_asteroid, axis=0)
                    ser = pd.Series(list(output_img))
                    images.append(ser, ignore_index=True)
            else:
                output_img = np.insert(output_img, 0, ycoord, axis=0)
                output_img = np.insert(output_img, 0, xcoord, axis=0)
                output_img = np.insert(output_img, 0, is_asteroid, axis=0)
                ser = pd.Series(list(output_img))
                images.append(ser, ignore_index=True)
        print(images)
        x_j += horizontal_rate
        y_j += horizontal_rate
    x_i = 0
    x_j = 100
    y_i += vertical_rate
    y_j += vertical_rate
```

coordinates

labels

data

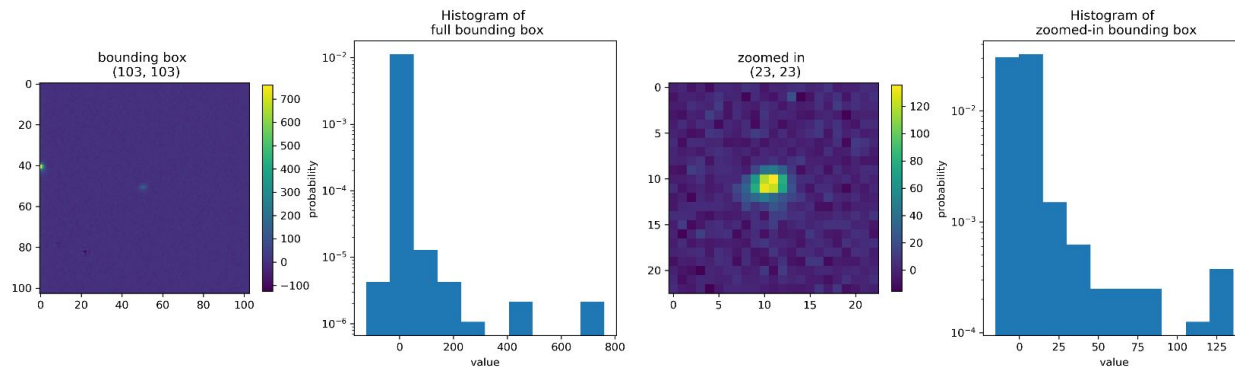
	0	1	2	3	4	5	6
0	1.0	440.0	1850.0	-3.824375	1.372986	-3.082868	-7.105562
1	1.0	2690.0	1250.0	-2.523140	-2.341650	0.950483	-5.158835
2	0.0	1020.0	50.0	-5838.244629	-5838.188965	-5838.499512	-5838.229980
3	1.0	880.0	850.0	2.191117	19.888529	-0.715346	1.965577
4	0.0	1760.0	1350.0	5.071833	-1.558900	3.726840	1.145760
...	...	...	...	...	...	...	...
8355	0.0	140.0	2550.0	-2210.338135	-2210.561279	-2210.546387	-2210.533447
8356	1.0	500.0	1850.0	5.963348	-4.030199	-3.084712	-1.785947
8357	0.0	2020.0	2350.0	-6.619294	6.354520	11.248178	-3.555027
8358	1.0	2070.0	2100.0	-2.727310	-4.862266	-2.805742	6.603151
8359	0.0	1110.0	1850.0	-5.737750	6.292427	-1.537311	-3.439410

- Snippet of data_generator

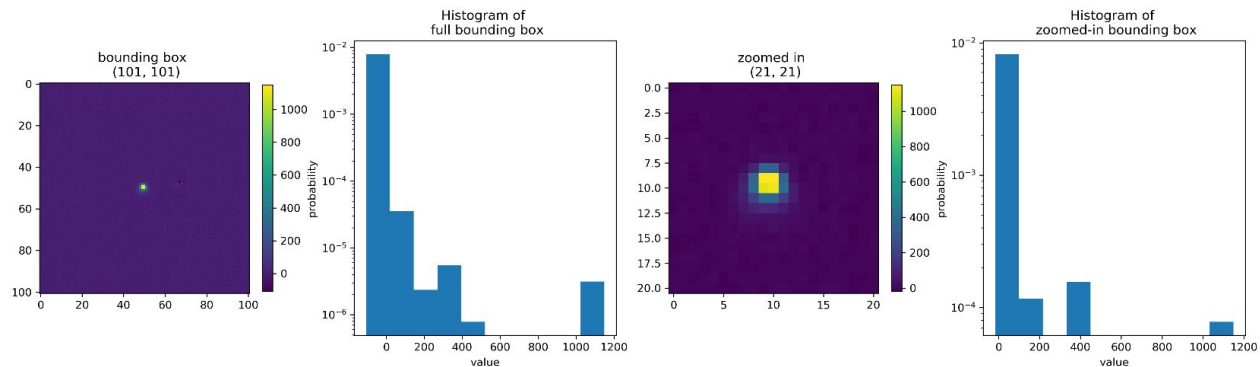
Challenge Problem: Asteroids Detection

EDA

Example 1

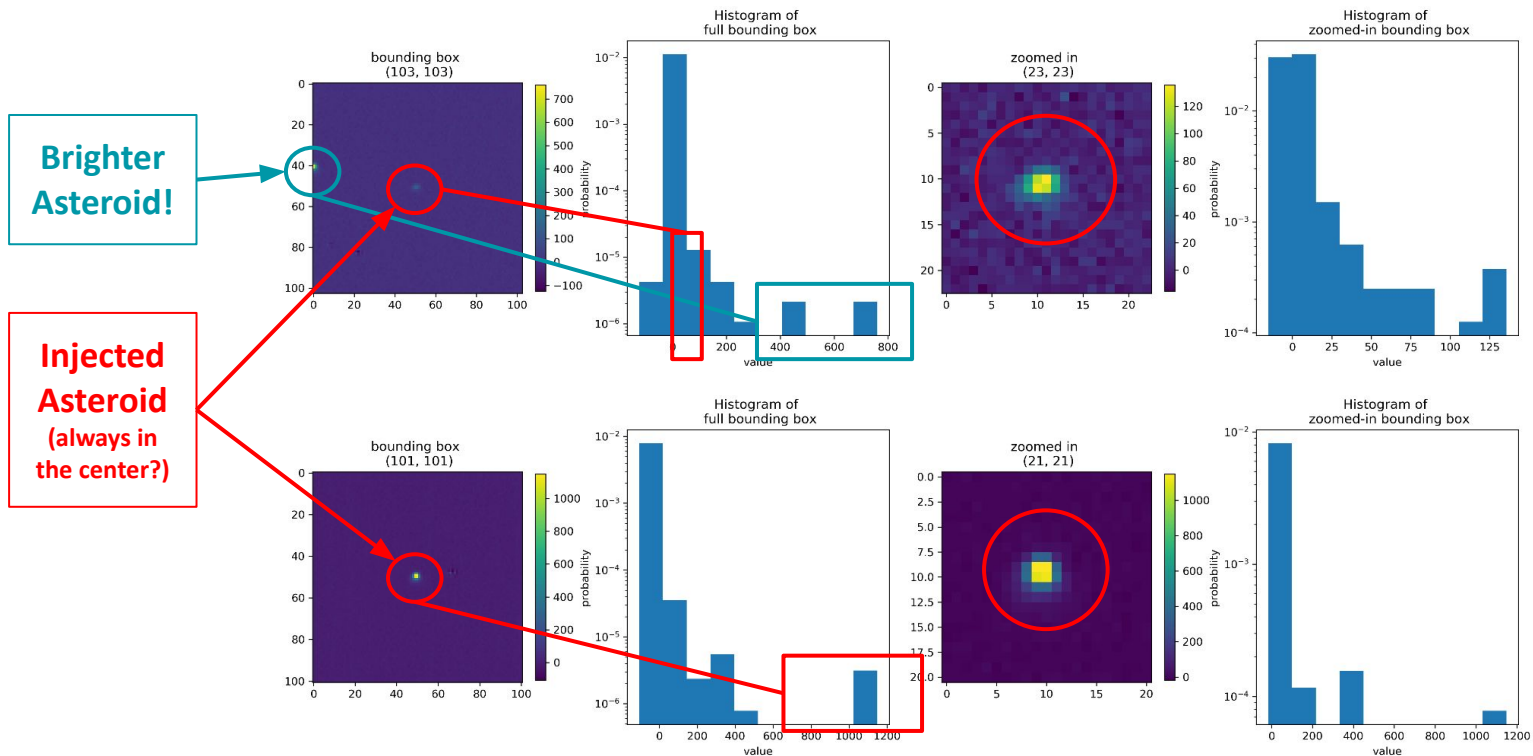


Example 2



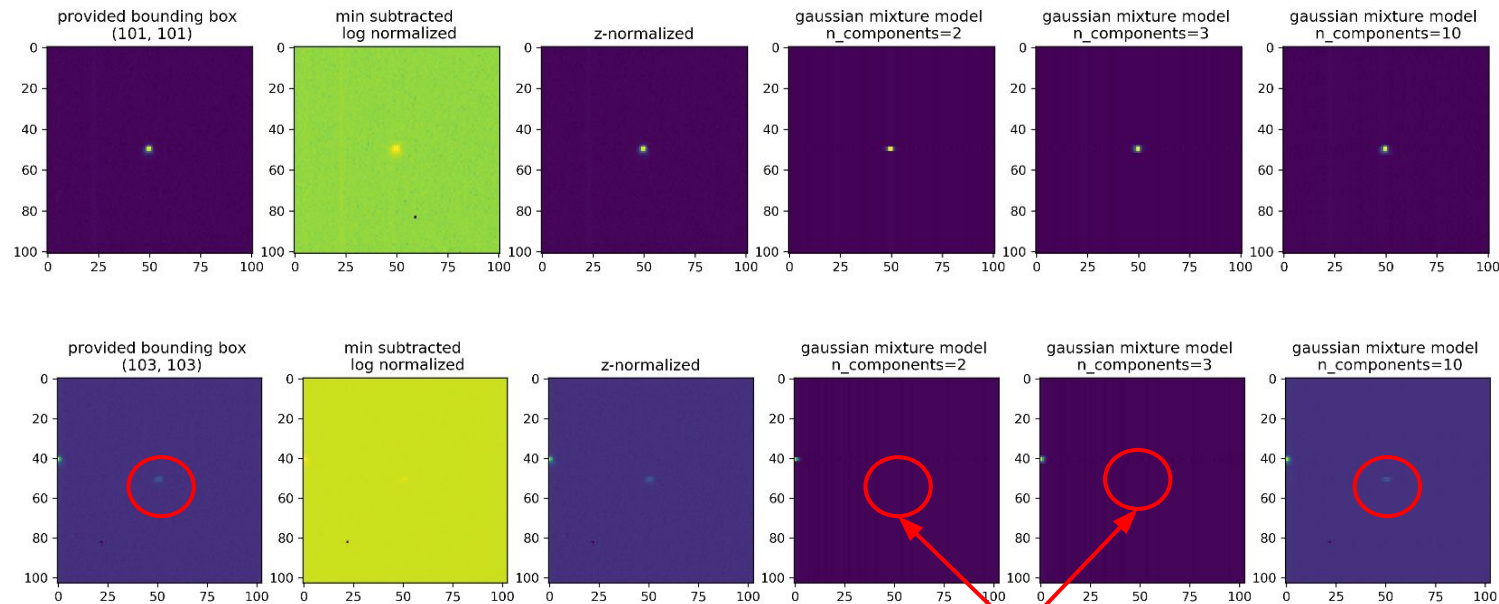
Challenge Problem: Asteroids Detection

EDA



Challenge Problem: Asteroids Detection

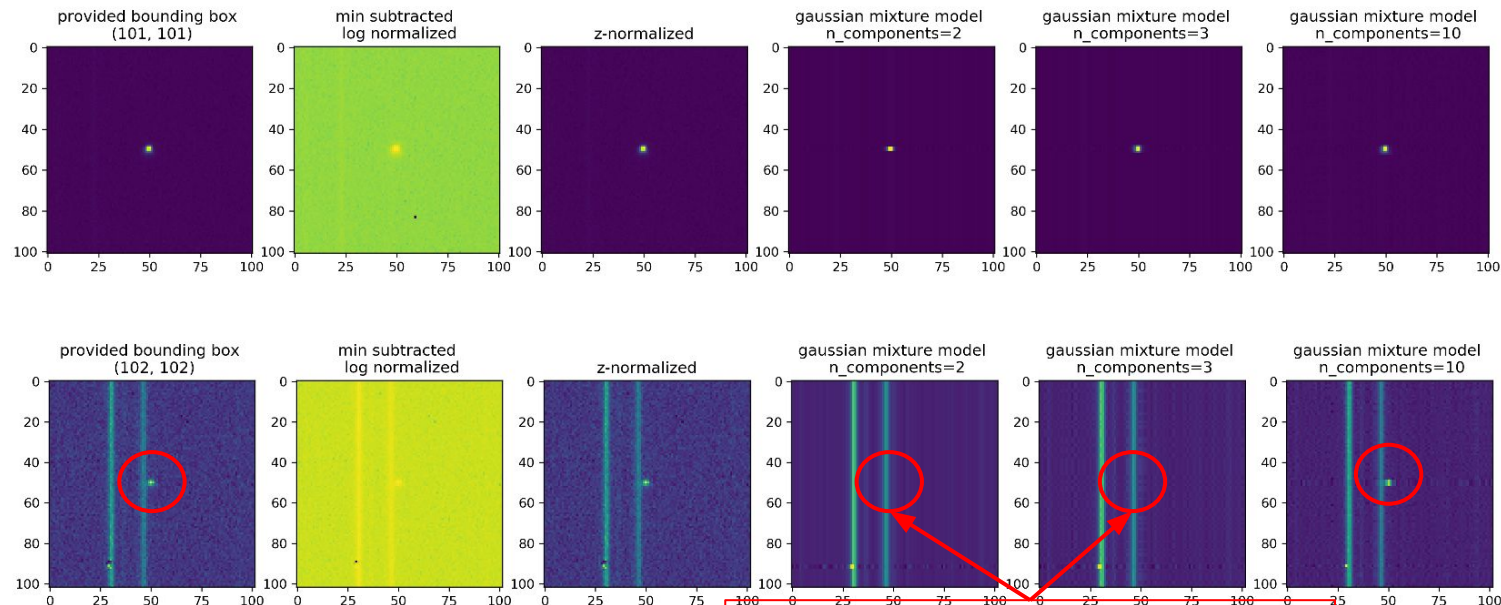
EDA Preprocessing



**Harsh preprocessing
loses fainter asteroid!**

Challenge Problem: Asteroids Detection

EDA Preprocessing



**Harsh preprocessing loses fainter
asteroid amid noise!
Very bad!**

Asteroids Detection

Pipeline

Dataset Preparation

- Generate asteroid samples (Class 1)
- **Generate background samples (Class 0)**
- Normalize data
- Split dataset
 - Train = 75%
 - Validation = 15%
 - Test = 10%
- Build Data Loader



Model Training

- Neural Networks



Model Evaluation

- Training loss
- Accuracy
- Precision-Recall
- F1-Score
- ROC Curve

Outline



Team Introductions



Background



Results

Task	Image Classification
Method	Simple CNN
Dataset	Asteroids Detection

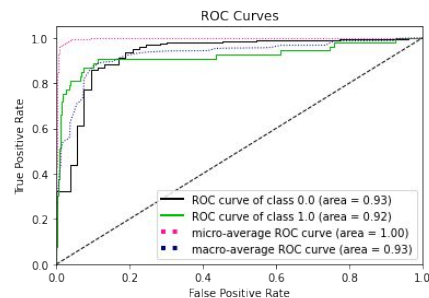
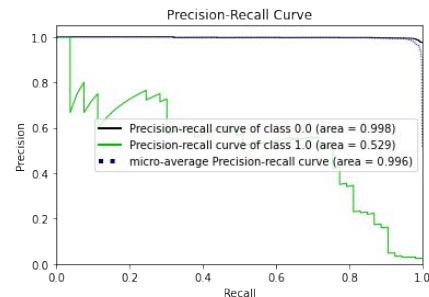
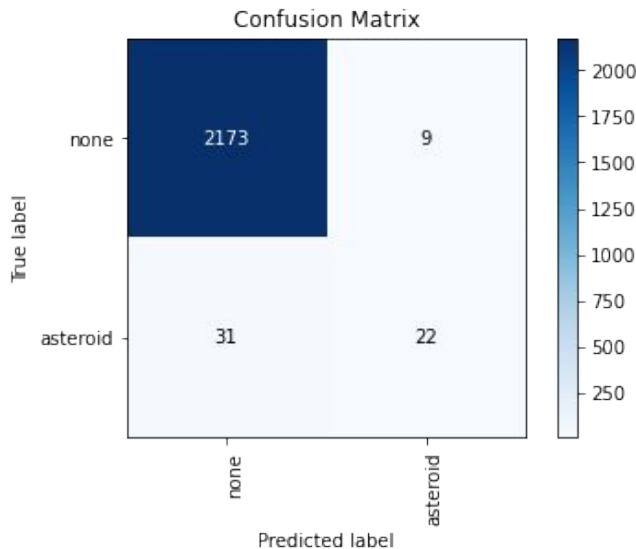


Summary

Challenge Problem: Asteroids Detection

Simple CNN Trained on 1 difference image

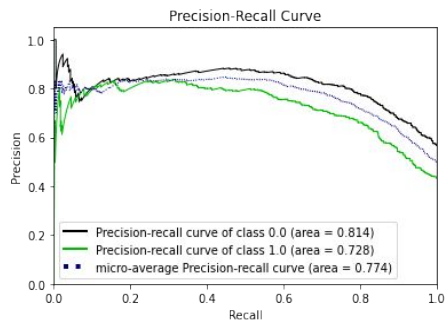
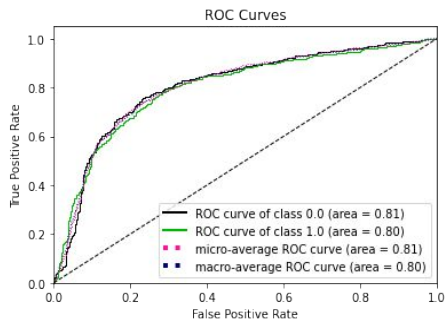
- Accuracy: .9959 (most of trained data is no-asteroid)
- Horizontal_rate = 10
- Vertical_rate = 100
- 20 asteroids total
- 50 epochs
- 100x100 images:



Challenge Problem: Asteroids Detection

Simple CNN

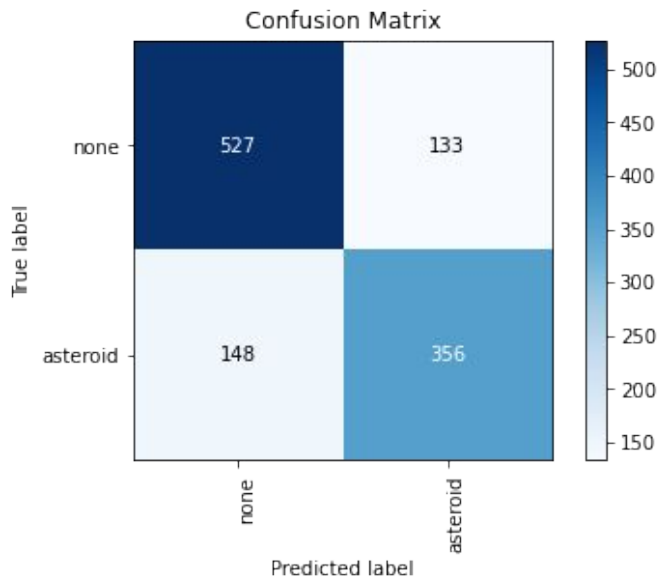
Trained on 10 difference images



- Trained on 10 difference images
- Cleaned data to be roughly 50% asteroid pictures
- Was this an anomaly? All images combined performed poorly (random guess level)

Issues

- Didn't clean data well enough
- Didn't automate cropping dead pixels
- Seemingly random borders & shapes with very high difference

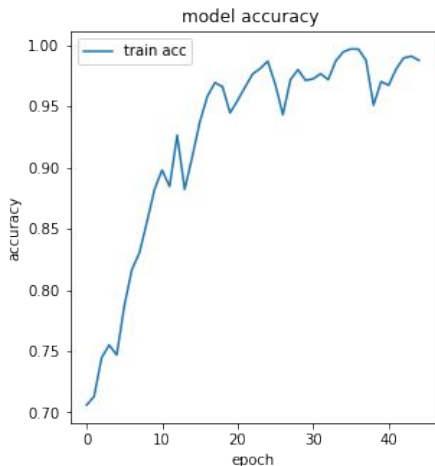


Challenge Problem: Asteroids Detection

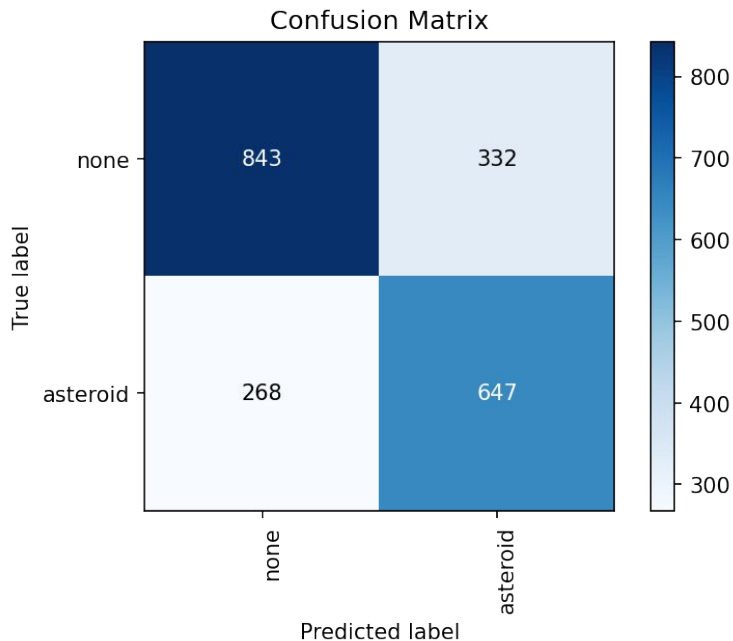
Simple CNN

Trained on 20 difference images

- Trained on 20 difference images
- Cleaned data to be roughly 50% asteroid pictures



- Found 600 asteroids in the test data

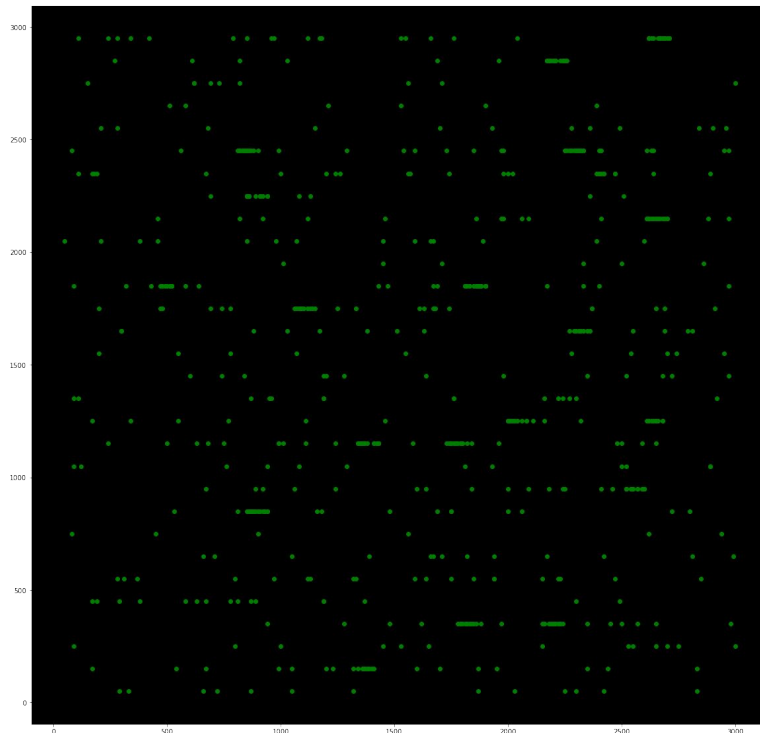


Overlap between prediction and true location of asteroids

- Location of bounding boxes where predicted is_asteroid == testing is_asteroid == 1
- Note: these aren't the exact location, just the location of the center of the bounding box

```
def get_xs(pred_y: pred_y, test_y: test_y):  
    xs = []  
    i = 0  
    while i < len(pred_y):  
        if pred_y[i] == test_y[i] == 1:  
            row = i + test_start  
            col = 1  
            xs.append(data2[i][1])  
        i += 1  
    return xs
```

Location of the
center of the
bounding box

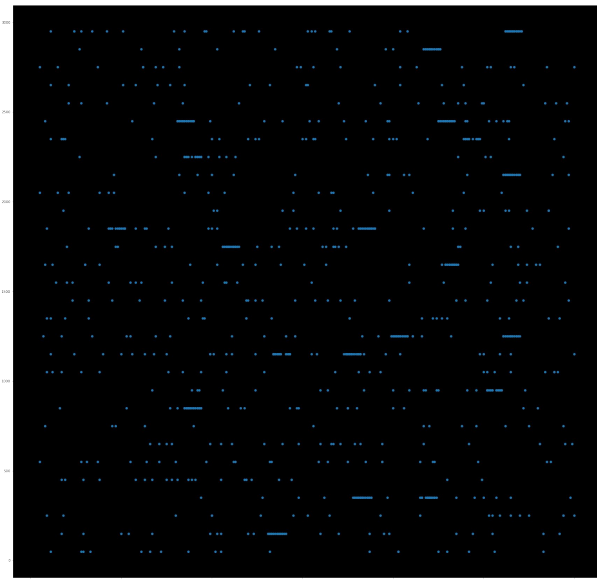


Challenge Problem:
Asteroids Detection

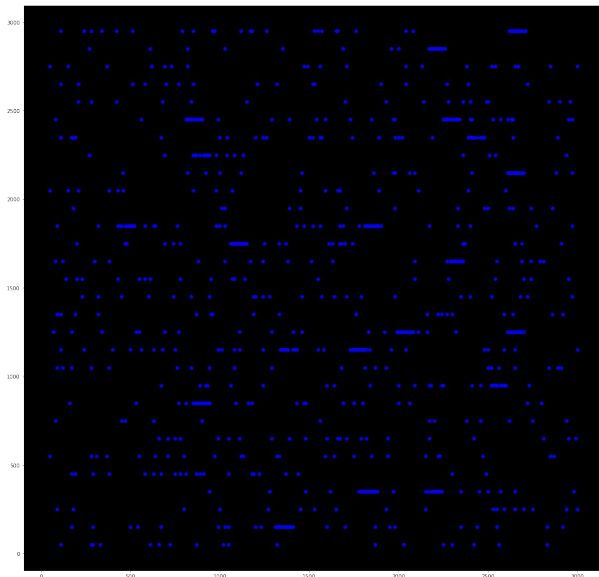
Simple CNN

Trained on 20 difference images

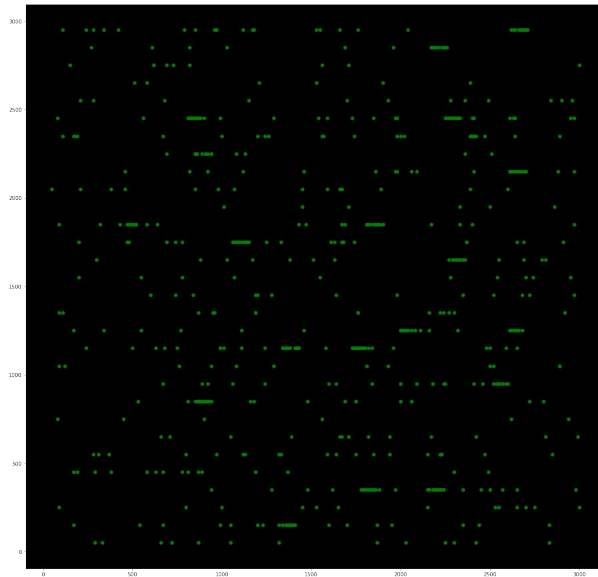
True



Predicted

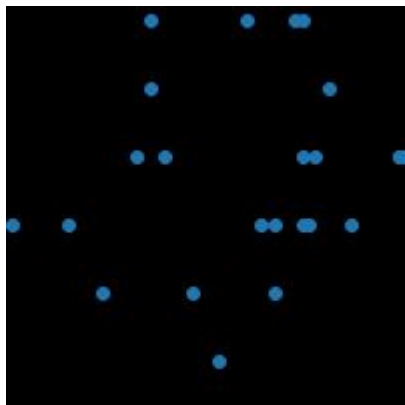


Overlap

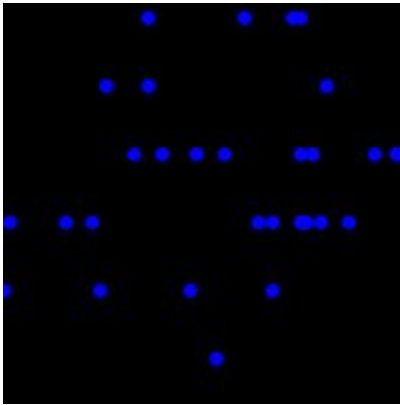


200x200 box in the center of the data

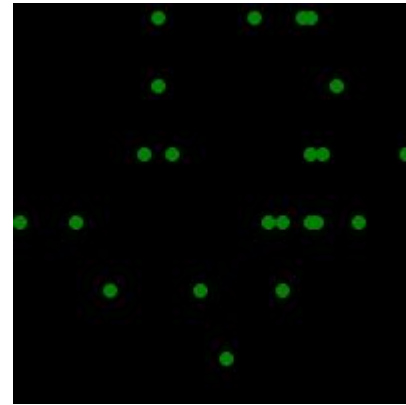
True



Predicted



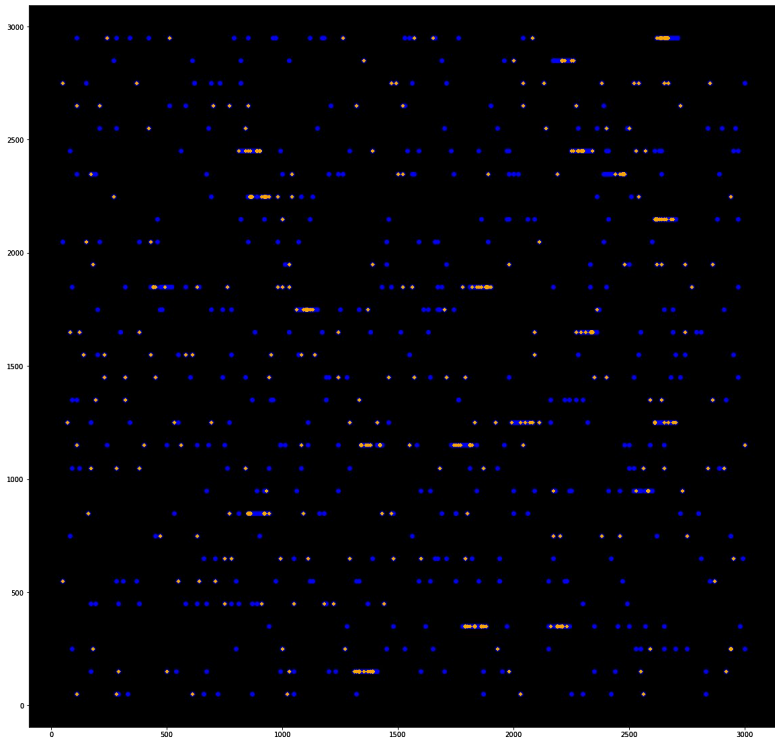
Overlap



Challenge Problem: Asteroids Detection

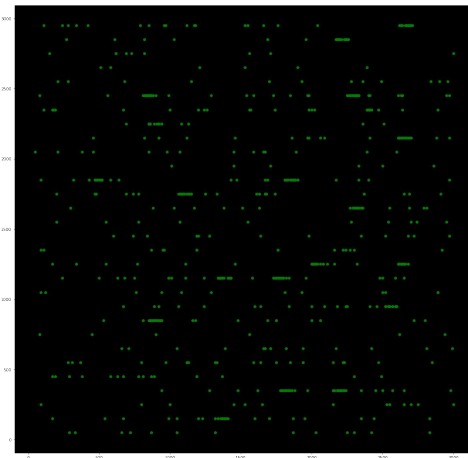
Simple CNN

Trained on 20 difference images



- Blue = predicted asteroid
- Orange = falsely predicted asteroid
- Green = correctly predicted asteroid

- ~650 of the 900 predictions were correct
- ~200 asteroids in the testing data were not predicted



Outline



Team Introductions



Background



Results

Task	Image Classification
Method	LeNet5
Dataset	Asteroids Detection



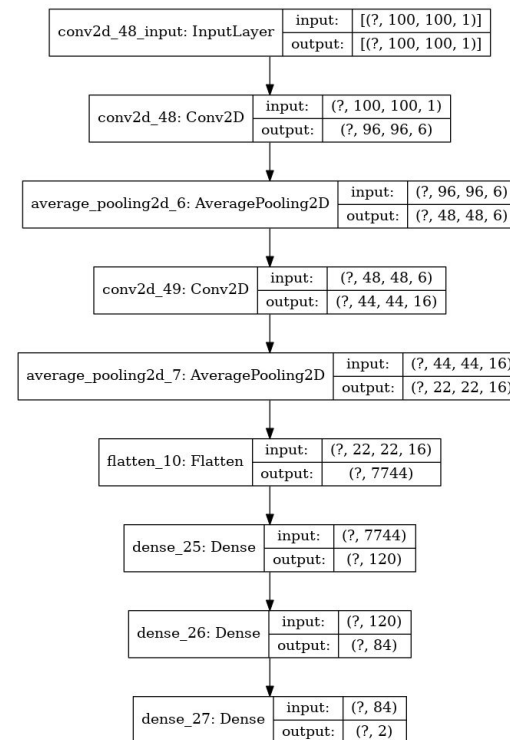
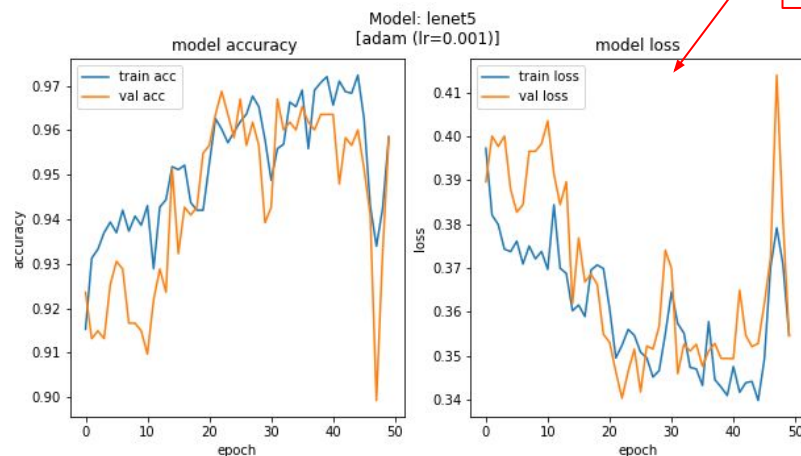
Summary

Challenge Problem: Asteroids Detection

DATASET

- 100 x 100 images
- Class 0: No Asteroids (n = 2000)
- Class 1: Asteroids (n = 2000)
- Train, Validation, Test = 75%, 15%, 10%

**Training needs
more work!
(lower learning rate)**



Challenge Problem: Asteroids Detection

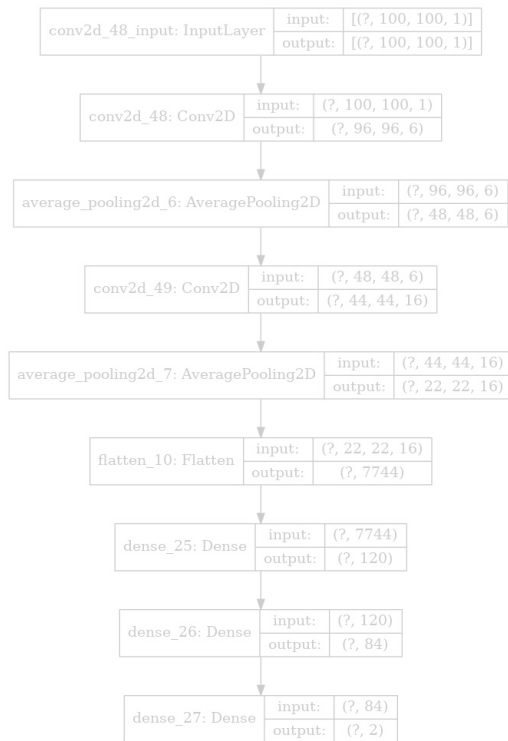
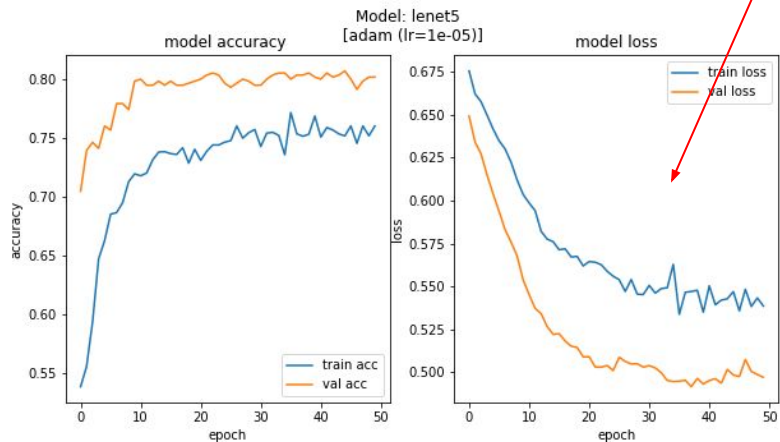
LeNet5

Data from 10 difference images

DATASET

- 100 x 100 images
- Class 0: No Asteroids (n = 2000)
- Class 1: Asteroids (n = 2000)
- Train, Validation, Test = 75%, 15%, 10%

That's a bit better...



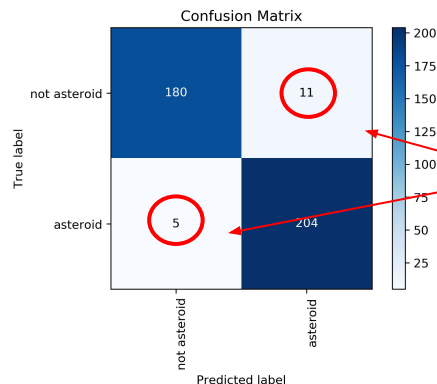
Challenge Problem: Asteroids Detection

Data from 10 difference images

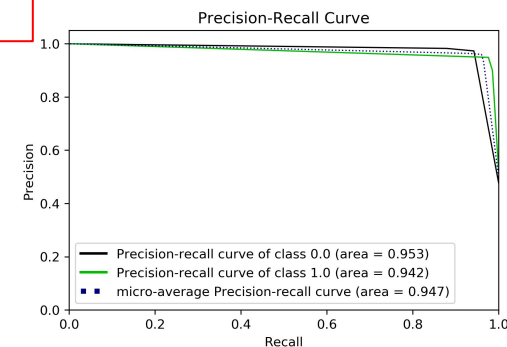
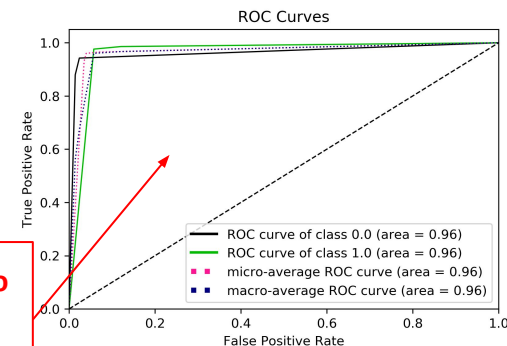
	precision	recall	f1-score	support
not asteroid (class 0)	0.97	0.94	0.957	191
asteroid (class 1)	0.948	0.97	0.96	209
accuracy				0.96

Good support
for both
classes

Shows we do
better than
random



Twice as
many false
positives as
false
negatives!

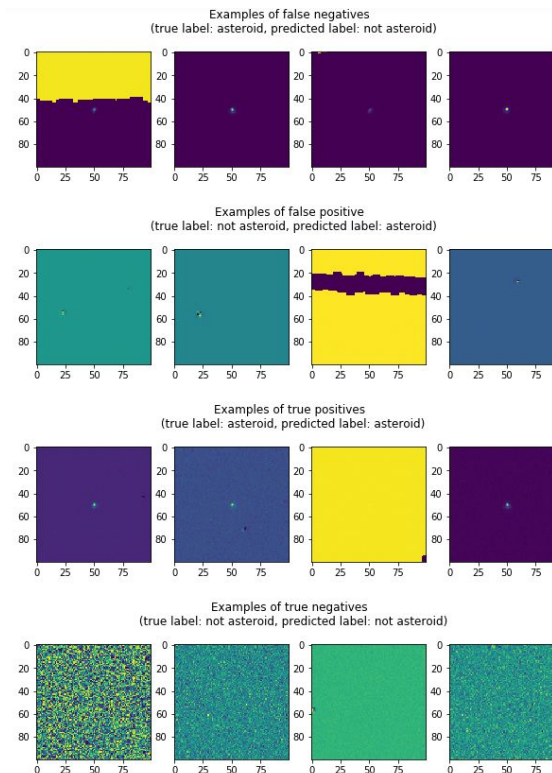
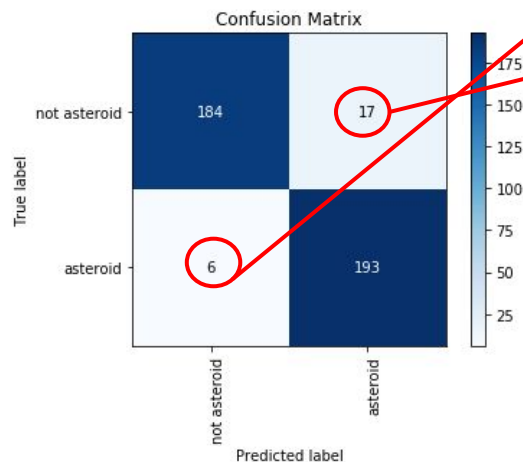


Challenge Problem: Asteroids Detection

Data from 10 difference images

DATASET

- 100 x 100 zoomed-in images
- Class 0: No Asteroids (n = 2000)
- Class 1: Asteroids (n = 2000)
- Train, Validation, Test = 75%, 15%, 10%



**Note: We can't be
sure of TRUE true
negatives!**

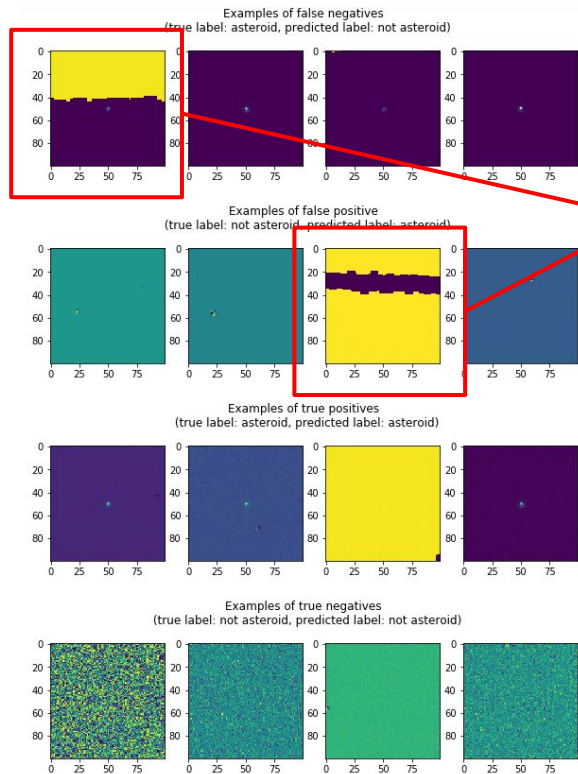
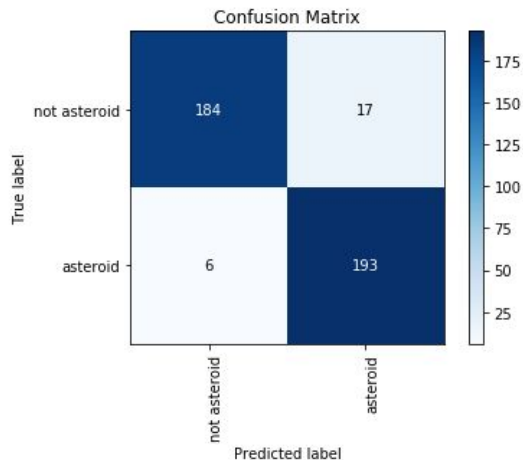
Challenge Problem: Asteroids Detection

LeNet5

Data from 10 difference images

DATASET

- 100 x 100 zoomed-in images
- Class 0: No Asteroids (n = 2000)
- Class 1: Asteroids (n = 2000)
- Train, Validation, Test = 75%, 15%, 10%



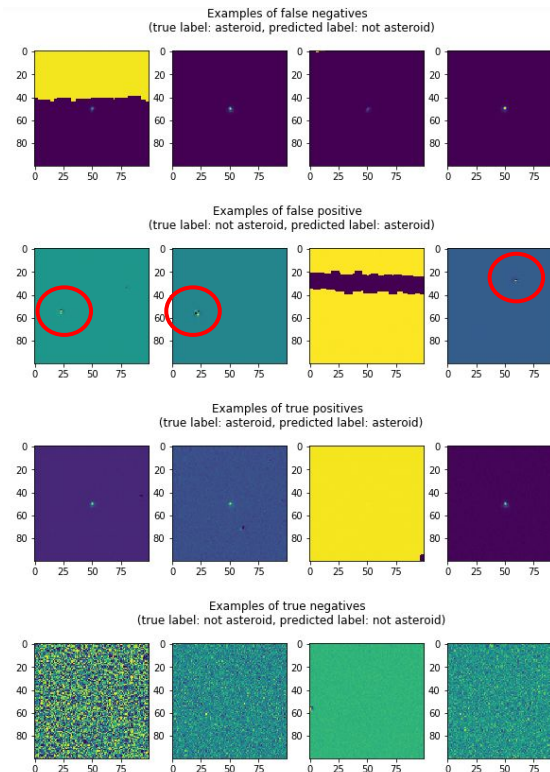
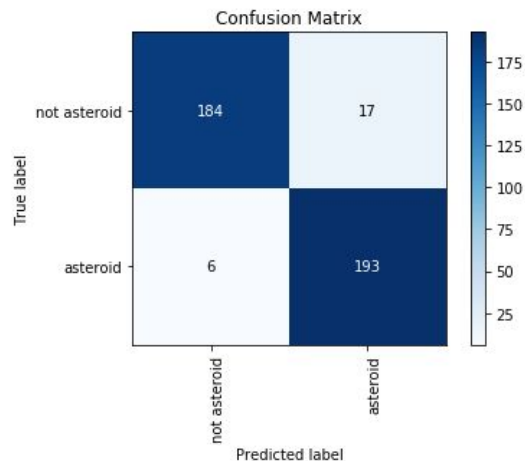
**Better data
cleaning
needed!**

BUT...

Challenge Problem: Asteroids Detection

DATASET

- 100 x 100 zoomed-in images
- Class 0: No Asteroids (n = 2000)
- Class 1: Asteroids (n = 2000)
- Train, Validation, Test = 75%, 15%, 10%



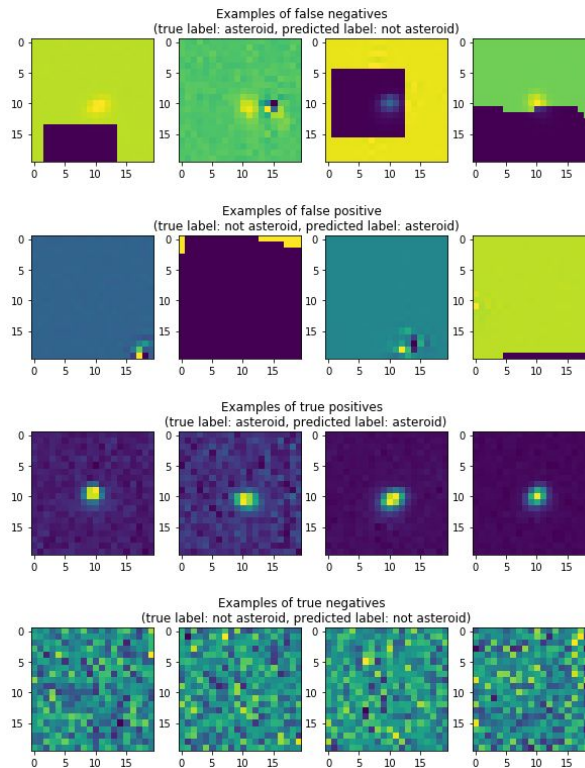
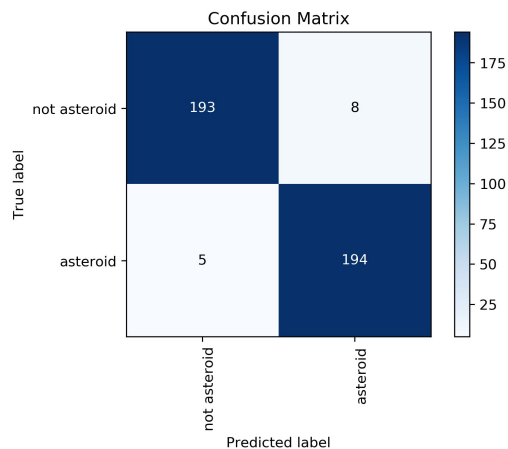
False Positives!
Are those hidden
asteroids?

Challenge Problem: Asteroids Detection

Data from 10 difference images

DATASET

- 20 x 20 zoomed-in images
- Class 0: No Asteroids (n = 2000)
- Class 1: Asteroids (n = 2000)
- Train, Validation, Test = 75%, 15%, 10%

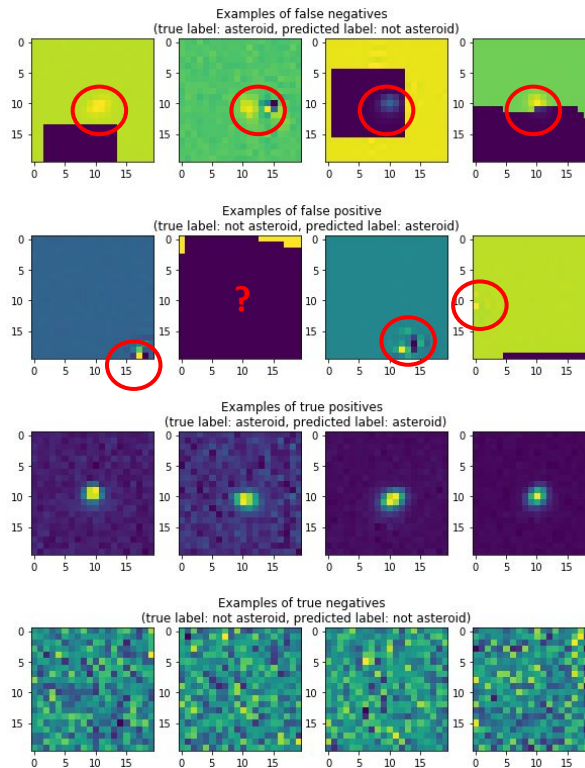
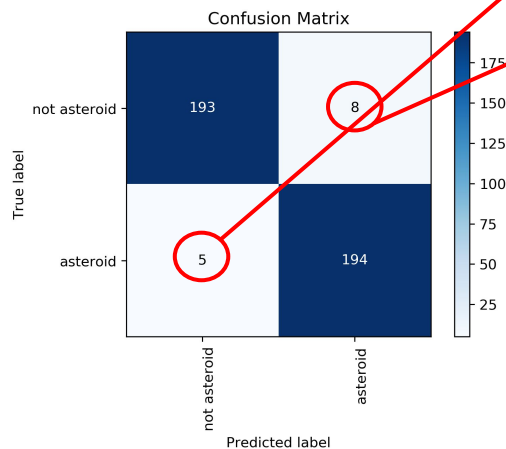


Challenge Problem: Asteroids Detection

Data from 10 difference images

DATASET

- 20 x 20 zoomed-in images
- Class 0: No Asteroids (n = 2000)
- Class 1: Asteroids (n = 2000)
- Train, Validation, Test = 75%, 15%, 10%



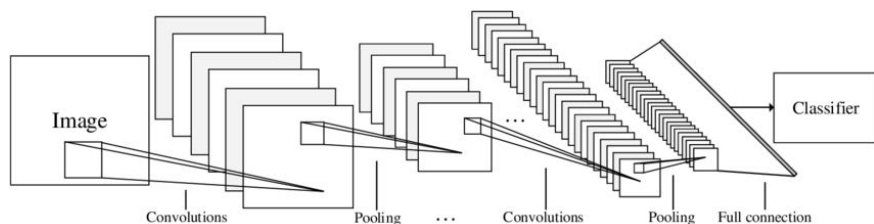
**Better data
cleaning
needed!**

Challenge Problem: Asteroids Detection

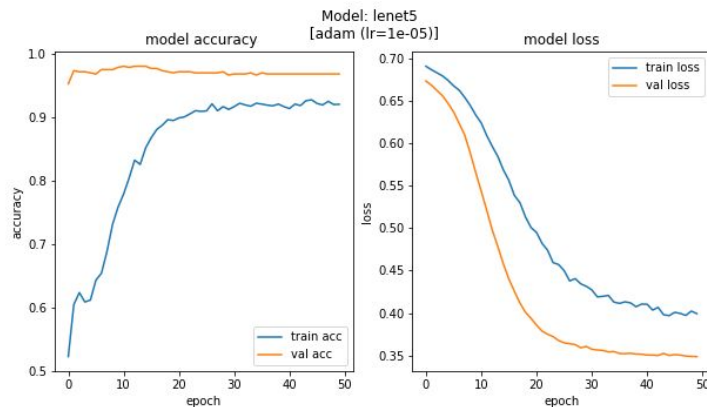
LeNet5

Caveats to note

Data: 20x20 images
($p=n=2000$)
75-15-10 split



LeNet5 is tiny!
20,706 trainable parameters!



Easier to train!

Outline



Team Introductions



Background



Results



Summary

Summary

Mini-Challenge Problem:

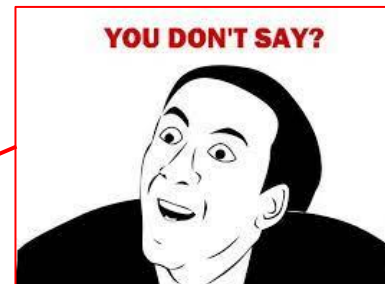
Star vs Galaxy Classification

- We trained neural networks for image classification as well as some classic ML models on vectorized data.
- Simple neural networks outperform classic models.
- It is 'easier' to classify stars than galaxies.

Challenge Problem:

Asteroid Detection

- We posed the asteroid detection problem as binary image classification.
- We generated negative samples from provided difference images.
- Simple neural networks performed well on a sample of provided data.
- **Data cleaning is crucial to avoid false positives and false negatives.**



Acknowledgements

We thank LLNL DSI for providing a challenging yet fun problem

Shout-outs:

- Brian Gallagher for being a patient and helpful organizer
- Jen Bellig ...well, she's basically awesome
- Ryan Dana for setting up the challenge problems and development environments
- Kerianne Pruett for being a great mentor





That's all Folks!

quod erat demonstrandum